EN.553.724 Probabilistic Machine Learning

Jonathan Y. Ma

June 2026

This course was taught by Dr. Holden Lee in the Spring 2026 Semester. The notes are transcribed by Jonathan Ma and may contain errors and omitted content, so any issues are to be directed towards me.

Contents

Chapter 1

Introduction

1.1 Probabilistic Modeling and Learning

1.1.1 Classical vs. Modern Methods

Classical probabilistic models (e.g. graphical models) and modern "deep" generative models differ primarily in how structure is specified.

- **Classical approaches**
  - Leverage domain knowledge to specify model structure.
  - Work well when the model is correctly specified.
  - Provide interpretable probabilistic quantities and uncertainty quantification.
- **Modern approaches**
  - Learn highly complex distributions directly from data.
  - Require large datasets.
  - Often lack interpretability.
- **Goal**

  combine structured probabilistic modeling with flexible function approximation.

1.1.2 Learning a Function

Model:

$$y = f(x) + \varepsilon, \quad \mathbb{E}[\varepsilon|x] = 0.$$

Given data $\{(x_i, y_i)\}_{i=1}^{N}$, choose f_θ.

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(x_i), y_i).$$

1.1.3 Maximum Likelihood Interpretation

Assume:

$$\varepsilon \sim \mathcal{N}(0, I).$$

Then:

$$p(y_i|x_i, \theta) \propto \exp\left(-\frac{1}{2}\left\|y_i - f_\theta(x_i)\right\|^2\right).$$

Proposition 1.1.1. *Minimizing squared loss is equivalent to maximum likelihood estimation.*

$$\arg\max_{\theta} p(y_{1:N}|x_{1:N}) = \arg\min_{\theta} \sum_{i=1}^{N} \left\|y_i - f_\theta(x_i)\right\|^2.$$

1.1.4 Learning a Distribution

Given $x_1, \ldots, x_N \overset{\text{iid}}{\sim} \mathbb{P}$:

$$\hat{\theta} = \arg\max_{\theta} \sum_{i=1}^{N} \log p_\theta(x_i) = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} -\log p_\theta(x_i).$$

1.2 Measure-Theoretic Foundations

1.2.1 Discrete and Continuous Probability

- Discrete:

$$\mathbb{P}(A) = \sum_{x \in A} p(x)$$

- Continuous:

$$\mathbb{P}(A) = \int_A p(x)\, \mathrm{d}x$$

Unified via:

$$\mathbb{P}(A) = \int_A p(x)\, \lambda(\mathrm{d}x).$$

1.2.2 Probability Spaces

Definition 1.2.1. A measurable space is $(\Omega, \mathcal{F})$.

Definition 1.2.2. A probability measure $\mathbb{P}$ satisfies:

- $\mathbb{P}(A) \geq 0$
- $\mathbb{P}(\Omega) = 1$
- countable additivity

If a density exists:

$$\mathbb{P}(A) = \int_A p(x)\, \lambda(\,\mathrm{d}x).$$

1.2.3 Expectation

$$\mathbb{E}[f] = \int f(x)\, \mathbb{P}(\,\mathrm{d}x)$$

If density exists:

$$\mathbb{E}[f] = \int f(x)p(x)\, \lambda(\,\mathrm{d}x).$$

1.2.4 Radon–Nikodym Derivative

Definition 1.2.3. $\mathbb{P} \ll Q$ if $Q(A) = 0 \Rightarrow \mathbb{P}(A) = 0$.

Theorem 1.2.1 (Radon–Nikodym)**.** *If* $\mathbb{P} \ll Q$*, then*

$$\int f\, \mathrm{d}\mathbb{P} = \int f \frac{\mathrm{d}\mathbb{P}}{\mathrm{d}Q}\, \mathrm{d}Q.$$

If densities exist:

$$\frac{\mathrm{d}\mathbb{P}}{\mathrm{d}Q}(x) = \frac{p(x)}{q(x)}.$$

1.2.5 Random Variables

Definition 1.2.4. A random variable is a measurable function $X : \Omega \to \mathbb{R}$.

Distribution:

$$\mathbb{P}_X(A) = \mathbb{P}(X \in A).$$

1.2.6 Conditional Probability

$$\mathbb{P}(A|B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$$

$$p(x|y) = \frac{p(x,y)}{p(y)}$$

$$p(x|y) = \frac{p(x)p(y|x)}{p(y)}$$

1.2.7 Beta-Binomial Model

$$\theta \sim \text{Beta}(\alpha, \beta), \quad X|\theta \sim \text{Binomial}(m+n, \theta).$$

Posterior:

$$\theta|X \sim \text{Beta}(\alpha+m, \beta+n).$$

1.2.8 Conditional Independence

Definition 1.2.5. $X \perp Y|Z$ if

$$p(x, y|z) = p(x|z)p(y|z).$$

1.3 Divergences and Information

1.3.1 Total Variation

Definition 1.3.1.

$$\text{TV}(\mathbb{P}, Q) = \frac{1}{2} \int |p-q| \, \lambda(\,\mathrm{d}x).$$

Proposition 1.3.1.

$$\text{TV}(\mathbb{P}, Q) = \max_A |\mathbb{P}(A) - Q(A)|.$$

1.3.2 f-Divergence

Definition 1.3.2.

$$D_f(\mathbb{P}\|Q) = \mathbb{E}_Q\left[f\left(\frac{p}{q}\right)\right].$$

$$\text{KL}(\mathbb{P}\|Q) = \int p \log \frac{p}{q}.$$

1.3.3 Non-Negativity

Theorem 1.3.1 (Jensen)**.**

$$\mathbb{E}[f(X)] \geq f(\mathbb{E}[X]).$$

Proposition 1.3.2.

$$D_f(\mathbb{P}\|Q) \geq 0.$$

Proof.

$$\mathbb{E}_Q f\left(\frac{p}{q}\right) \geq f\left(\mathbb{E}_Q \frac{p}{q}\right) = f(1) = 0.$$

□

1.3.4 Entropy and KL

Definition 1.3.3.

$$H(\mathbb{P}) = -\int p \log p.$$

$$\mathrm{KL}(\mathbb{P}\|Q) = -H(\mathbb{P}) - \mathbb{E}_{\mathbb{P}}[\log q].$$

1.3.5 MLE and KL

Proposition 1.3.3.

$$\mathbb{E}_{\mathbb{P}}[-\log p_\theta(x)] = \mathrm{KL}(\mathbb{P}\|\mathbb{P}_\theta) + H(\mathbb{P}).$$

$$\Rightarrow \mathrm{MLE} = \arg\min_\theta \mathrm{KL}(\mathbb{P}\|\mathbb{P}_\theta).$$

1.3.6 KL Direction

$$\mathrm{KL}(\mathbb{P}\|Q) \quad \text{mass-covering}$$

$$\mathrm{KL}(Q\|\mathbb{P}) \quad \text{mode-seeking}$$

1.3.7 Invariance

Proposition 1.3.4. *For invertible g:*

$$D_f(\mathbb{P}\|Q) = D_f(g_\# \mathbb{P}\|g_\# Q).$$

1.4 Optimal Transport and Integral Probability Metrics

1.4.1 Couplings and Wasserstein Distance

Definition 1.4.1. Let $(\Omega, \mathcal{F})$ be a measurable space with metric d, and let μ, ν be probability measures on Ω.

A *coupling* of μ and ν is a probability measure γ on $\Omega \times \Omega$ such that:

$$\gamma(A \times \Omega) = \mu(A), \quad \gamma(\Omega \times A) = \nu(A).$$

Interpretation: a coupling specifies a joint distribution (X, Y) with marginals $X \sim \mu$, $Y \sim \nu$.

Definition 1.4.2. The Wasserstein-p distance is defined as

$$W_p(\mu, \nu) = \left(\inf_{\gamma \in \Gamma(\mu,\nu)} \mathbb{E}_{(X,Y)\sim\gamma}[d(X, Y)^p] \right)^{1/p},$$

where $\Gamma(\mu, \nu)$ is the set of all couplings of μ, ν.

Remark 1.4.1. W_1 is often called the *earth mover's distance*, since it measures the minimal cost of transporting mass from μ to ν.

1.4.2 Integral Probability Metrics

Definition 1.4.3. Let $\mathcal{F}$ be a class of functions $f : \Omega \to \mathbb{R}$. The integral probability metric (IPM) is

$$d_{\mathcal{F}}(\nu, \mu) = \sup_{f \in \mathcal{F}} \left| \mathbb{E}_\nu[f] - \mathbb{E}_\mu[f] \right|.$$

Lemma 1.4.1. *Let $\mathcal{F} = \{f : \|f\|_\infty \leq 1\}$. Then*

$$\mathrm{TV}(\mu, \nu) = \frac{1}{2} d_{\mathcal{F}}(\nu, \mu).$$

Proof. For any f with $\|f\|_\infty \leq 1$,

$$|\mathbb{E}_\nu[f] - \mathbb{E}_\mu[f]| = \left| \int f(x)(\nu - \mu)(\,\mathrm{d}x) \right| \leq \int |f(x)|\, |\nu - \mu|(\,\mathrm{d}x) \leq \int |\nu - \mu|(\,\mathrm{d}x).$$

Taking the supremum gives

$$d_{\mathcal{F}}(\nu, \mu) \leq 2\,\mathrm{TV}(\mu, \nu).$$

Equality is achieved by choosing

$$f(x) = \mathrm{sign}(p(x) - q(x)),$$

which yields

$$\mathbb{E}_\nu[f] - \mathbb{E}_\mu[f] = \int |p(x) - q(x)|\,\mathrm{d}x.$$

□

1.4.3 Kantorovich–Rubinstein Duality

Theorem 1.4.1 (Kantorovich–Rubinstein)**.** *Let $\mathcal{F}$ be the set of 1-Lipschitz functions on $\mathbb{R}^d$. Then*

$$W_1(\mu, \nu) = d_{\mathcal{F}}(\nu, \mu).$$

Sketch. For any coupling γ and any 1-Lipschitz f,

$$|f(x) - f(y)| \leq d(x, y).$$

Thus

$$\mathbb{E}_\nu[f] - \mathbb{E}_\mu[f] = \mathbb{E}_\gamma[f(Y) - f(X)] \leq \mathbb{E}_\gamma[d(X, Y)].$$

Taking infimum over γ gives

$$d_{\mathcal{F}}(\nu, \mu) \leq W_1(\mu, \nu).$$

The reverse inequality follows by constructing an optimal dual potential. □

Remarks on Metrics vs Divergences

- Total variation and Wasserstein are *metrics.*
- f-divergences (e.g. KL) are not metrics:
  - not symmetric
  - do not satisfy triangle inequality
- IPMs provide a unifying view of distances via function classes.

1.5 Maximum Entropy and Exponential Families

1.5.1 Maximum Entropy Principle

Let P be an unknown distribution on Ω. Suppose we are given moment constraints:

$$\mathbb{E}_P[\phi_i(X)] = \mu_i, \quad i = 1, \dots, d,$$

or equivalently

$$\mathbb{E}_P[\phi(X)] = \mu \in \mathbb{R}^d,$$

where $\phi : \Omega \to \mathbb{R}^d$.

Remark 1.5.1. These constraints alone do not uniquely determine P.

A natural principle is to choose the *least informative* distribution satisfying the constraints.

Definition 1.5.1. The maximum entropy distribution (with respect to a reference measure ν) is

$$\arg\max_P \; H_\nu(P) \quad \text{s.t.} \quad \mathbb{E}_P[\phi] = \mu,$$

where

$$H_\nu(P) = -\int p(x) \log p(x)\, \nu(\,\mathrm{d}x).$$

Derivation via Lagrange Multipliers

Assume Ω is finite for simplicity, and write $p(x) = P(X = x)$.

We maximize

$$-\sum_{x \in \Omega} p(x) \log p(x)$$

subject to:

$$\sum_x p(x) = 1, \quad \sum_x p(x)\phi_i(x) = \mu_i.$$

Define the Lagrangian:

$$\mathcal{L}(p, \lambda_0, \lambda) = -\sum_x p(x) \log p(x) + \lambda_0 \left(\sum_x p(x) - 1 \right) + \sum_{i=1}^{d} \lambda_i \left(\sum_x p(x)\phi_i(x) - \mu_i \right).$$

Taking derivatives:

$$\frac{\partial \mathcal{L}}{\partial p(x)} = -\log p(x) - 1 + \lambda_0 + \sum_{i=1}^{d} \lambda_i \phi_i(x).$$

Setting to zero:

$$\log p(x) = \lambda_0 - 1 + \sum_{i=1}^{d} \lambda_i \phi_i(x).$$

Thus

$$p(x) \propto \exp\left(\sum_{i=1}^{d} \lambda_i \phi_i(x)\right) = \exp(\langle \theta, \phi(x) \rangle),$$

where $\theta = (\lambda_1, \ldots, \lambda_d)$.

This gives the exponential family form.

1.5.2 Exponential Family

Definition 1.5.2. Let $(\mathcal{X}, \nu)$ be a measure space and $\phi : \mathcal{X} \to \mathbb{R}^d$.

The exponential family is

$$p_\theta(x) = \frac{\exp(\langle \theta, \phi(x) \rangle)}{Z(\theta)} = \exp\Big(\langle \theta, \phi(x) \rangle - A(\theta)\Big),$$

where

$$Z(\theta) = \int \exp(\langle \theta, \phi(x) \rangle)\, \nu(\,\mathrm{d}x), \quad A(\theta) = \log Z(\theta).$$

Remark 1.5.2.

- θ is the natural parameter
- $\phi(x)$ are sufficient statistics
- $A(\theta)$ is the log-partition function

1.5.2.1 Example: Gaussian from Exponential Family

Let $\phi(x) = (x, x^2)$ on $\mathbb{R}$.

Then

$$p_\theta(x) \propto \exp(\theta_1 x + \theta_2 x^2).$$

Completing the square:

$$\theta_1 x + \theta_2 x^2 = \theta_2 \left(x + \frac{\theta_1}{2\theta_2}\right)^2 - \frac{\theta_1^2}{4\theta_2}.$$

For $\theta_2 < 0$, this is a Gaussian:

$$p_\theta(x) = \mathcal{N}(\mu, \sigma^2), \quad \mu = -\frac{\theta_1}{2\theta_2}, \quad \sigma^2 = -\frac{1}{2\theta_2}.$$

1.5.3 Maximum Entropy Characterization

Theorem 1.5.1. *If there exists θ such that*

$$\mathbb{E}_{p_\theta}[\phi(X)] = \mu,$$

then p_θ is the maximum entropy distribution among all distributions satisfying the moment constraints.

Proof. Let P satisfy $\mathbb{E}_P[\phi] = \mu$. Then

$$\mathrm{KL}(P \| p_\theta) = \mathbb{E}_P[\log p(x) - \log p_\theta(x)] \geq 0.$$

Rearranging:

$$-H(P) \geq \mathbb{E}_P[\log p_\theta(x)].$$

Using exponential family form:

$$\mathbb{E}_P[\log p_\theta(x)] = \mathbb{E}_P[\langle \theta, \phi(x) \rangle - A(\theta)] = \langle \theta, \mu \rangle - A(\theta).$$

Thus

$$H(P) \leq A(\theta) - \langle \theta, \mu \rangle = H(p_\theta),$$

so p_θ maximizes entropy. □

1.5.4 Mean Parameter Mapping

Define the mapping

$$\nabla A(\theta) = \mathbb{E}_{p_\theta}[\phi(X)].$$

Definition 1.5.3. Let

$$\mathcal{M} = \{\mu \in \mathbb{R}^d : \exists P \text{ such that } \mathbb{E}_P[\phi(X)] = \mu\}$$

be the set of achievable mean parameters.

Definition 1.5.4. The statistics $\{\phi_i\}_{i=1}^d$ are *minimal* if

$$\{1, \phi_1, \ldots, \phi_d\} \text{ are linearly independent.}$$

Proposition 1.5.1. *If ϕ is minimal, then:*

- $\nabla A : \Theta \to \mathcal{M}$ *is injective*
- $\nabla A : \Theta \to \mathcal{M}^\circ$ *is surjective (onto the interior)*

Sketch. If ϕ is minimal, then $A(\theta)$ is strictly convex:

$$\nabla^2 A(\theta) = \mathrm{Cov}_{p_\theta}(\phi(X)) \succ 0.$$

Strict convexity implies injectivity of ∇A.

Surjectivity onto the interior follows from convex duality and properties of cumulant generating functions. □

1.5.5 Convexity and Geometry

Proposition 1.5.2. *The log-partition function satisfies:*

$$\nabla^2 A(\theta) = \mathrm{Cov}_{p_\theta}(\phi(X)) \succeq 0.$$

Remark 1.5.3.

- $A(\theta)$ is convex
- strictly convex if ϕ is minimal
- exponential families form a convex dual pair:

$$\theta \leftrightarrow \mu = \nabla A(\theta)$$

Non-Minimal Case

If ϕ is not minimal, then there exists $v \neq 0$ such that

$$\langle v, \phi(x) \rangle = c \quad \forall x.$$

Then

$$p_{\theta+tv}(x) = p_\theta(x),$$

so the parametrization is not identifiable, and ∇A is not injective.

Chapter 2

Graphical Models

2.1 Directed Graphical Models (Bayesian Networks)

2.1.1 Factorization in DAGs

Definition 2.1.1. Let $G = (V, E)$ be a directed acyclic graph (DAG), and let $\{X_v\}_{v \in V}$ be random variables.

A distribution P *factorizes according to* G if

$$p(x_V) = \prod_{v \in V} p\Big(x_v \mid x_{\mathrm{pa}(v)}\Big),$$

where $\mathrm{pa}(v)$ denotes the set of parents of node v.

Remark 2.1.1. Each factor $p(x_v \mid x_{\mathrm{pa}(v)})$ is a *local conditional model.*

Definition 2.1.2. A *topological ordering* of a DAG is an ordering $(v_1, \ldots, v_n)$ such that

$$v_i \to v_j \implies i < j.$$

Remark 2.1.2. This allows a generative interpretation:

$$X_{v_1} \to X_{v_2} \to \cdots \to X_{v_n},$$

sampling each variable conditioned on its parents.

2.1.2 Independence Structure

Definition 2.1.3. Let $S_v = \Big(\{v\} \cup \mathrm{Desc}(v) \cup \mathrm{pa}(v)\Big)^c$.

A DAG encodes the independence relations

$$X_v \perp X_{S_v} \mid X_{\mathrm{pa}(v)}.$$

Proposition 2.1.1. *A distribution factorizes according to G if and only if it satisfies the independence relations encoded by G.*

2.1.3 Factorization ⇒ Independence (Key Idea)

Lemma 2.1.1. *If P factorizes according to a DAG G, then for any upward-closed set U,*

$$p(x_U) = \prod_{u \in U} p(x_u \mid x_{\mathrm{pa}(u)}).$$

Sketch. Sum out variables not in U using the factorization; terms outside U cancel due to normalization. □

This leads to conditional independence:

$$X_v \perp X_{S_v} \mid X_{\mathrm{pa}(v)}.$$

2.1.4 Constructing a DAG

Given variables $X_1, \ldots, X_n$:

- Choose an ordering $X_1, \ldots, X_n$
- For each k, choose a minimal set $S \subseteq \{1, \ldots, k-1\}$ such that

$$X_k \perp X_{[k-1] \setminus S} \mid X_S$$

- Set $\mathrm{pa}(k) = S$

Remark 2.1.3. The resulting graph is not unique.

Remark 2.1.4. Causality can guide graph construction, but cannot generally be inferred from the graph alone.

2.1.5 Compactness of Representation

For n variables taking values in $\{1, \ldots, r\}$:

- Full joint distribution requires:

$$r^n - 1 \quad \text{parameters}$$

- If max indegree is d:

$$O(nr^{d+1})$$

  parameters suffice

Remark 2.1.5. Graphical models exploit conditional independence for exponential savings.

2.1.6 Examples

2.1.6.1 Markov Chain

$$X_{k+1} \perp X_1, \ldots, X_{k-1} \mid X_k$$

Factorization:

$$p(x_1, \ldots, x_n) = p(x_1) \prod_{k=2}^{n} p(x_k \mid x_{k-1})$$

2.1.6.2 d-th Order Markov Model

$$X_{k+d} \perp X_1, \ldots, X_{k-1} \mid X_k, \ldots, X_{k+d-1}$$

2.1.6.3 Naive Bayes

Latent class C:

$$X_k \perp X_{-k} \mid C$$

Factorization:

$$p(c, x_1, \ldots, x_n) = p(c) \prod_{k=1}^{n} p(x_k \mid c)$$

2.1.7 Latent Variable Models

Definition 2.1.4. A *latent variable model* introduces hidden variables Z such that

$$p(x) = \int p(x, z) \, \mathrm{d}z.$$

Remark 2.1.6. Latent variables simplify structure by introducing conditional independence.

2.1.7.1 Mixture Model

$$z_i \sim \text{Multinomial}(\pi), \quad x_i \mid z_i \sim \mathcal{N}(\mu_{z_i}, \Sigma_{z_i})$$

2.1.7.2 Stochastic Block Model

$$z_i \sim \text{Multinomial}(\pi), \quad A_{ij} \mid z_i, z_j \sim \text{Bernoulli}(p_{z_i z_j})$$

2.1.7.3 Noisy-OR Model

$$x_j \mid z \sim \text{Bernoulli}\left(1 - \exp\left(-\sum_i w_{ij} z_i\right)\right)$$

2.1.7.4 Hidden Markov Model (HMM)

$$z_t = Az_{t-1} + \epsilon_t, \quad x_t = Cz_t + \eta_t$$

Remark 2.1.7. This is a linear Gaussian state-space model.

2.1.8 d-Separation

Definition 2.1.5. A path in a DAG is *blocked* by a set C if one of the following holds:

1. Chain: $X \to Z \to Y$ or $X \leftarrow Z \leftarrow Y$, and $Z \in C$
2. Fork: $X \leftarrow Z \to Y$, and $Z \in C$
3. Collider: $X \to Z \leftarrow Y$, and $Z \notin C$ and no descendant of Z is in C

Definition 2.1.6. A and B are *d-separated* by C if all paths between them are blocked.

Theorem 2.1.1. *If P factorizes according to G and A, B are d-separated by C, then*

$$X_A \perp X_B \mid X_C.$$

2.2 Undirected Graphical Models (Markov Random Fields)

2.2.1 Factorization

Definition 2.2.1. Let $G = (V, E)$ be an undirected graph. Let $\mathcal{C}$ be the set of maximal cliques.

A distribution factorizes over G if

$$p(x) \propto \exp(-E(x)), \quad E(x) = \sum_{C \in \mathcal{C}} \phi_C(x_C),$$

or equivalently

$$p(x) = \prod_{C \in \mathcal{C}} \psi_C(x_C).$$

Remark 2.2.1. $\psi_C = e^{-\phi_C}$ are called *clique potentials*.

2.2.2 Markov Properties

Definition 2.2.2. P satisfies:

- Global Markov property:

$$X_A \perp X_B \mid X_C \quad \text{if } C \text{ separates } A, B$$

- Local Markov property:
$$X_v \perp X_{V\setminus(\{v\}\cup N(v))} \mid X_{N(v)}$$

- Pairwise Markov property:
$$X_v \perp X_w \mid X_{V\setminus\{v,w\}}, \quad (v,w) \notin E$$

Remark 2.2.2. Under positivity conditions:
$$\text{global} \iff \text{local} \iff \text{pairwise}$$

2.2.3 Energy-Based Models

Definition 2.2.3. An energy-based model defines
$$p_\theta(x) \propto e^{-E_\theta(x)}.$$

Remark 2.2.3.
- Lower energy $\Rightarrow$ higher probability
- $E_\theta(x) = \langle \theta, \phi(x) \rangle$ recovers exponential families

2.2.4 Example: Ising Model

$$p(x) \propto \exp\left(\sum_i h_i x_i + \sum_{(i,j)\in E} J_{ij} x_i x_j\right), \quad x_i \in \{-1, 1\}$$

Conditional distribution:
$$p(x_i = 1 \mid x_{-i}) = \frac{\exp(h_i + \sum_{j\in N(i)} J_{ij} x_j)}{\exp(h_i + \sum_{j\in N(i)} J_{ij} x_j) + \exp(-h_i - \sum_{j\in N(i)} J_{ij} x_j)}.$$

Remark 2.2.4. Unlike DAGs, sampling is generally difficult due to the global partition function.

2.3 Additional Properties of Independence

Intersection Property

Proposition 2.3.1. *Suppose $p(x, y, z, w)$ has positive density with respect to a product measure. Then the* intersection property *holds:*
$$\begin{cases} X \perp Y \mid Z, W, \\ X \perp W \mid Z, Y \end{cases} \quad \Rightarrow \quad X \perp (Y, W) \mid Z.$$

Remark 2.3.1. This property does *not* hold in general without positivity.

Proposition 2.3.2. *If the intersection property holds, then the global, local, and pairwise Markov properties are equivalent.*

Hammersley–Clifford Theorem

Theorem 2.3.1 (Hammersley–Clifford)**.** *Let $G = (V, E)$ be an undirected graph. Suppose P has positive density.*

Then:

$$P \text{ satisfies the Markov properties} \iff P \text{ factorizes over } G.$$

Remark 2.3.2. Positivity is essential: without it, independence does not imply factorization.

Canonical Representation

Theorem 2.3.2. *Any strictly positive distribution can be written as*

$$p(x) = \exp\left(\sum_{S \subseteq V} \xi_S(x_S)\right),$$

where

$$\xi_S(x_S) = \sum_{Z \subseteq S} (-1)^{|S|-|Z|} \log p(x_Z, x_{S^c} = 0).$$

Remark 2.3.3.

- This is an inclusion–exclusion type expansion
- If P factorizes over G, then $\xi_S = 0$ for non-cliques S

2.4 Factor Graphs

Definition 2.4.1. A *factor graph* is a bipartite graph with:

- variable nodes V
- factor nodes F

and edges only between variables and factors.

A distribution factorizes as

$$p(x_V) = \prod_{a \in F} \varphi_a(x_{N(a)}),$$

where $N(a)$ are variables connected to factor a.

Remark 2.4.1. Factor graphs unify:

- DAG factorizations
- MRF factorizations

2.4.1 Example: Restricted Boltzmann Machine (RBM)

Let visible units $v \in \{0,1\}^{d_v}$ and hidden units $h \in \{0,1\}^{d_h}$.

$$p(v,h) \propto \exp\left(\langle v, Wh\rangle + \langle b, v\rangle + \langle a, h\rangle\right)$$

Remark 2.4.2. Key property:

$$p(h \mid v) = \prod_j p(h_j \mid v), \quad p(v \mid h) = \prod_i p(v_i \mid h)$$

Remark 2.4.3. This conditional independence enables efficient Gibbs sampling.

2.4.2 Example: Coding Theory (LDPC)

$$p(x,z) \propto \prod_i \mathbb{1}\{H_i z = 0\} \prod_j p(x_j \mid z_j)$$

Remark 2.4.4. Inference corresponds to belief propagation.

2.5 Conditional Random Fields

Definition 2.5.1. Let $(V \cup W, E)$ be an undirected graph.

A *conditional random field (CRF)* is a conditional distribution

$$P(X_W \mid X_V) \propto \prod_{C \in \mathcal{C}} \psi_C(x_C),$$

where cliques C involve variables in W.

Remark 2.5.1. CRFs model conditional structure without specifying a joint model over (V, W).

2.6 Directed $\to$ Undirected Conversion

Definition 2.6.1. The *moralized graph* $\mathcal{M}[G]$ of a DAG G is obtained by:

- adding edges between all pairs of parents of each node
- dropping edge directions

Proposition 2.6.1. *If P factorizes according to a DAG G, then it factorizes according to the undirected graph $\mathcal{M}[G]$.*

2.7 Independence via Moralization

Theorem 2.7.1. *Let G be a DAG and A, B, C disjoint sets. The following are equivalent:*

1. *A and B are d-separated by C in G*
2. *A and B are separated by C in the moralized ancestral graph*

Moreover, if P factorizes according to G, then

$$X_A \perp X_B \mid X_C.$$

Definition 2.7.1. The *ancestral graph* is the subgraph induced by A, B, C and their ancestors.

2.8 Markov Blanket

Definition 2.8.1. A set U is a *Markov blanket* for v if

$$X_v \perp X_{V \setminus (U \cup \{v\})} \mid X_U.$$

Proposition 2.8.1. *1. In an undirected graph, $N(v)$ is a Markov blanket.*

2. In a DAG, the Markov blanket is:

$$\mathrm{pa}(v) \cup \mathrm{ch}(v) \cup \text{co-pa}(v).$$

Remark 2.8.1. The Markov blanket gives the minimal conditioning set needed to isolate a node.

2.9 Learning Graph Structure (Undirected Case)

Proposition 2.9.1. *Assume P is positive. Define graph G by:*

$$(v, w) \in E \iff X_v \not\perp X_w \mid X_{V \setminus \{v,w\}}.$$

Then G is the unique minimal graph for which P satisfies the pairwise Markov property.

Comparison: Directed vs Undirected Models

- Directed models:
  - capture causal / generative structure
  - sampling is straightforward

    - require ordering / domain knowledge

- Undirected models:
    - symmetric relationships
    - easier structure learning
    - inference often harder (partition function)

2.10 Inference Problems

2.10.1 Core Tasks

Given a probabilistic model $p_\theta(x)$:

- **Sampling:**
$$x \sim p_\theta(x)$$
- **Marginals:**
$$p_\theta(x_i) = \int p_\theta(x)\, \nu^{\otimes(n-1)}(dx_{-i})$$
- **MAP (maximum a posteriori):**
$$x^* = \arg\max_x p_\theta(x)$$
- **Partition function:**
$$Z(\theta) = \int \psi_\theta(x)\, \nu^{\otimes n}(dx)$$

Remark 2.10.1. These tasks are tightly coupled: computing one often requires solving the others.

Latent Variable Models

For joint models:
$$p_\theta(x, z) \propto \psi_\theta(x, z)$$

posterior inference requires:
$$p_\theta(z|x) = \frac{\psi_\theta(x, z)}{Z(\theta, x)}, \quad Z(\theta, x) = \int \psi_\theta(x, z)\, \nu_Z(dz)$$

- Sampling: $z \sim p_\theta(z|x)$
- Marginals: $p_\theta(z_i|x)$
- MAP: $\arg\max_z p_\theta(z|x)$

2.11 Computational Complexity

Complexity Classes

- $O(n)$, $O(n \log n)$, $O(n^c)$: polynomial time
- $O(2^n)$: exponential time

Definition 2.11.1. A *decision problem* outputs YES/NO.

- **P**: solvable in polynomial time
- **NP**: solutions verifiable in polynomial time
- **NP-complete**: hardest problems in NP
- **NP-hard**: at least as hard as NP-complete

Example 2.11.1. 3-SAT: given a Boolean formula, does a satisfying assignment exist?

Counting Problems

Definition 2.11.2. #P: counting versions of NP problems (e.g., number of satisfying assignments).

Remark 2.11.1. Computing partition functions and marginals is typically #P-hard.

Theorem 2.11.1. *Computing partition functions or marginals in general graphical models is #P-hard.*

Remark 2.11.2. Even approximate counting (e.g., #SAT) can be NP-hard.

2.12 Approximation Algorithms

Definition 2.12.1. A *polynomial-time randomized approximation scheme (PRAS)* for Z outputs $\widehat{Z}$ such that

$$e^{-\epsilon} Z \leq \widehat{Z} \leq e^{\epsilon} Z$$

in polynomial time, with high probability.

Definition 2.12.2.
- **FPRAS**: runtime polynomial in n and $1/\epsilon$
- **PTAS**: deterministic approximation scheme

Equivalence: Sampling, Marginals, Partition Function

Definition 2.12.3. A family of problems is *self-reducible* if fixing variables reduces the problem to a smaller instance.

Theorem 2.12.1 (informal)**.** *For distributions $p_\theta(x) \propto \psi_\theta(x)$ on finite spaces:*

1. *Approximate sampling $\Rightarrow$ approximate marginals*
2. *Approximate marginals $\Rightarrow$ approximate sampling (if self-reducible)*
3. *Marginals $\Leftrightarrow$ partition function*

Theorem 2.12.2. *If we can sample within TV distance ϵ in $\mathrm{poly}(n, 1/\epsilon)$ time, then we can approximate marginals via empirical averages:*

$$\widehat{p}(x_i = a) = \frac{1}{N} \sum_{k=1}^{N} \mathbb{1}\{x_i^{(k)} = a\}$$

Proof. By Hoeffding's inequality:

$$\mathbb{P}\left(|\widehat{p} - p| \geq \delta\right) \leq 2\exp(-2N\delta^2)$$

so $N = O(1/\delta^2)$ suffices. $\square$

2.13 Exact Inference

2.13.1 Variable Elimination

Algorithm 1 Variable Elimination

Input: $p(x) \propto \prod_a \psi_a(x_{\partial a})$
Choose elimination order
for variable i **do**
  Multiply all factors involving i
  Sum out i
  Introduce new factor
end for
Output desired marginal

Remark 2.13.1. Intermediate factors grow in size $\to$ exponential complexity.

Treewidth

Definition 2.13.1. The *treewidth* $\mathrm{tw}(G)$ is the size of the largest intermediate factor (minus one) in optimal elimination ordering.

Remark 2.13.2. Complexity of variable elimination:

$$O(n \cdot \mathrm{state}^{\mathrm{tw}(G)+1})$$

2.14 Belief Propagation

2.14.1 Message Passing

Messages:

- Variable → factor:
$$m_{i\to a}(x_i) \propto \prod_{b\in\partial i\setminus a} m_{b\to i}(x_i)$$
- Factor → variable:
$$m_{a\to i}(x_i) \propto \sum_{x_{\partial a\setminus i}} \psi_a(x_{\partial a}) \prod_{j\in\partial a\setminus i} m_{j\to a}(x_j)$$

Marginals

$$p(x_i) \propto \prod_{a\in\partial i} m_{a\to i}(x_i)$$

$$p(x_{\partial a}) \propto \psi_a(x_{\partial a}) \prod_{j\in\partial a} m_{j\to a}(x_j)$$

2.14.2 Properties

- Exact on trees
- Equivalent to dynamic programming
- Linear-time in number of nodes (for trees)

Remark 2.14.1. On loopy graphs:

- becomes *loopy belief propagation*
- no guarantees, but often works well

2.15 Extensions

- Conditioning: add indicator factors
- Continuous variables: approximate messages (e.g., Gaussian)
- Junction tree algorithm: exact inference via tree decomposition

2.16 Takeaways

- Exact inference is exponential in general
- Tree structure enables efficient computation
- Message passing = dynamic programming on graphs
- Sampling, marginals, and partition function are deeply connected

Correctness of Belief Propagation (Trees)

We formalize what the messages compute.

Let $T_{b\to i}$ denote the subtree rooted at b when edge (b,i) is removed.

Proposition 2.16.1. *For a tree-structured factor graph:*

$$Z_{b\to i}(x_i) = \sum_{x_{V(T_{b\to i})\setminus i}} \prod_{c\in F(T_{b\to i})} \psi_c(x_{\partial c}),$$

and similarly

$$Z_{a\to j}(x_j) = \sum_{x_{V(T_{a\to j})\setminus j}} \prod_{c\in F(T_{a\to j})} \psi_c(x_{\partial c}).$$

Proof. By induction on the size of the subtree.

Base case: leaf node. Only one factor contributes, and the expression holds trivially.

Inductive step: assume the claim holds for all subtrees of depth $\leq k$. Then for a larger subtree,

$$Z_{a\to i}(x_i) = \sum_{x_{\partial a\setminus i}} \psi_a(x_{\partial a}) \prod_{j\in \partial a\setminus i} Z_{j\to a}(x_j),$$

which matches the factor-to-variable message update.

Thus the recursion exactly computes subtree marginals. □

Max-Product Belief Propagation (MAP)

Replace sums by maxima.

- Variable $\to$ factor:

$$M_{i\to a}(x_i) = \prod_{b\in \partial i\setminus a} M_{b\to i}(x_i)$$

- Factor $\to$ variable:

$$M_{a\to i}(x_i) = \max_{x_{\partial a\setminus i}} \psi_a(x_{\partial a}) \prod_{j\in \partial a\setminus i} M_{j\to a}(x_j)$$

Remark 2.16.1. This computes max-marginals:

$$M_i(x_i) \propto \max_{x_{-i}} p(x)$$

Algorithm 2 Max-product belief propagation

```
Initialize messages
repeat
    for factor a do
        update M_{a→i}
    end for
    for variable i do
        update M_{i→a}
    end for
until convergence
Output arg max_x p(x) from max-marginals
```

2.16.1 Hidden Markov Models (HMMs)

Model

Latent chain:

$$z_1 \to z_2 \to \cdots \to z_n, \quad x_t | z_t$$

Factorization:

$$p(z_{1:n}, x_{1:n}) = p(z_1) \prod_{t=2}^{n} p(z_t | z_{t-1}) \prod_{t=1}^{n} p(x_t | z_t)$$

Forward Pass

Define

$$f_t(i) = p(z_t = i, x_{1:t})$$

Recursion:

$$f_{t+1}(j) = p(x_{t+1} | z_{t+1} = j) \sum_i f_t(i)\, p(z_{t+1} = j | z_t = i)$$

Backward Pass

$$b_t(i) = p(x_{t+1:n} | z_t = i)$$

Recursion:

$$b_t(i) = \sum_j p(z_{t+1} = j | z_t = i)\, p(x_{t+1} | z_{t+1} = j)\, b_{t+1}(j)$$

Posterior

$$p(z_t = i|x_{1:n}) \propto f_t(i)\, b_t(i)$$

Viterbi Algorithm (MAP)

Define

$$\delta_t(j) = \max_{z_{1:t-1}} p(z_{1:t}, x_{1:t})$$

Recursion:

$$\delta_{t+1}(j) = p(x_{t+1}|z_{t+1} = j) \max_i \delta_t(i)\, p(z_{t+1} = j|z_t = i)$$

Remark 2.16.2. Viterbi is max-product BP on a chain.

2.16.2 Loopy Belief Propagation

- Same updates as BP, but on graphs with cycles
- No guarantees of convergence or correctness
- Often works well in practice

Example 2.16.1. LDPC decoding uses loopy BP.

Summary: VE vs BP

- **Variable elimination:**
  - works on any graph
  - computes one marginal at a time
  - complexity depends on treewidth
- **Belief propagation:**
  - exact on trees
  - computes all marginals simultaneously
  - linear-time for trees
  - approximate on loopy graphs

2.17 Takeaways

- Exact inference is exponential in general
- Trees $\Rightarrow$ efficient dynamic programming

- Message passing unifies many algorithms:
  - forward-backward
  - Viterbi
  - BP
- MAP inference = max-product BP

2.18 Sampling with Markov Chains

Problem Formulation

We want to approximately sample from a distribution

$$p(x) \propto e^{f(x)}, \quad x \in \Omega,$$

within ε in total variation distance.

Remark 2.18.1. This problem appears across:

- Bayesian inference
- statistical physics
- combinatorics / theoretical CS
- deep generative modeling

Remark 2.18.2. Sampling is essentially equivalent to computing expectations:

$$\mathbb{E}_{x\sim p}[f(x)] = \int f(x)p(x)\,dx,$$

which becomes intractable in high dimensions.

2.18.1 Monte Carlo Estimation

Given samples $X_1, \ldots, X_N \sim p$,

$$\mathbb{E}[f(X)] \approx \frac{1}{N}\sum_{i=1}^{N} f(X_i).$$

Proposition 2.18.1.

$$\mathrm{Var}\left(\frac{1}{N}\sum_{i=1}^{N} f(X_i)\right) = \frac{\mathrm{Var}(f(X))}{N}.$$

Proof. Uses independence:

$$\mathrm{Var}\left(\frac{1}{N}\sum f(X_i)\right) = \frac{1}{N^2}\sum \mathrm{Var}(f(X_i)) = \frac{1}{N}\mathrm{Var}(f(X)).$$

□

What Distributions Are Easy to Sample?

- Uniform distributions
- Finite discrete distributions
- 1D distributions via inverse CDF
- Product distributions:
$$p(x_1, \dots, x_n) = \prod_{i=1}^{n} p_i(x_i)$$
- Directed graphical models:
$$p(x) = \prod_{i=1}^{n} p(x_i | x_{\text{parents}(i)})$$
- Gaussian distributions

Remark 2.18.3. General high-dimensional distributions are not directly sampleable.

2.18.2 Importance Sampling

Let $Y \sim Q$.

Definition 2.18.1.
$$\widehat{\mu} = \frac{1}{N} \sum_{i=1}^{N} \frac{p(Y_i)}{q(Y_i)} f(Y_i).$$

Proposition 2.18.2.
$$\mathbb{E}_Q[\widehat{\mu}] = \mathbb{E}_P[f(X)].$$

Proof.
$$\mathbb{E}_Q\left[\frac{p(Y)}{q(Y)} f(Y)\right] = \int \frac{p(y)}{q(y)} f(y) q(y)\, dy = \int f(y) p(y)\, dy.$$

□

2.18.3 Rejection Sampling

Algorithm 3 Rejection sampling

Input: $q(x)$, c such that $p(x) \leq cq(x)$
repeat
  Sample $Y \sim Q$
  Sample $U \sim \text{Uniform}(0, 1)$
until $U \leq \frac{p(Y)}{cq(Y)}$
Output Y

Theorem 2.18.1. *If $p(x) \leq cq(x)$, then rejection sampling produces samples from p, and*

$$\#trials \sim \mathrm{Geom}(1/c).$$

Remark 2.18.4. Fails in high dimensions due to exponential scaling of c.

2.19 Markov Chains

Definition 2.19.1. A sequence (X_t) is a Markov chain if

$$X_{t+1} \perp X_1, \ldots, X_{t-1} \mid X_t,$$

i.e.,

$$p(x_{t+1}|x_1, \ldots, x_t) = p(x_{t+1}|x_t).$$

Definition 2.19.2. A Markov chain is time-homogeneous if

$$p(x_{t+1}|x_t) \text{ does not depend on } t.$$

Transition Kernels

Definition 2.19.3. A Markov kernel K is a function such that:

- $K(x, \cdot)$ is a probability measure
- measurable in x

For a distribution μ,

$$(\mu K)(A) = \int K(x, A)\, \mu(dx).$$

If densities exist:

$$(\mu K)(y) = \int \mu(x) k(x, y)\, dx.$$

Operator View

Definition 2.19.4. For $f : \Omega \to \mathbb{R}$,

$$Kf(x) = \mathbb{E}_{Y \sim K(x,\cdot)}[f(Y)] = \int f(y) K(x, dy).$$

Remark 2.19.1. K acts:

- on distributions: $\mu \mapsto \mu K$
- on functions: $f \mapsto Kf$

2.19.1 Stationary Distributions

Definition 2.19.5. π is stationary if

$$\pi K = \pi.$$

Remark 2.19.2. Then $\pi K^t = \pi$ for all t.

2.19.2 Conditions for Convergence

Definition 2.19.6. A Markov chain is:

- irreducible: any state can reach any other
- aperiodic: no deterministic cycles

Theorem 2.19.1. *If a finite Markov chain is irreducible and aperiodic:*

- *it has a unique stationary distribution* π
- $\mu_0 K^t \to \pi$

Theorem 2.19.2 (Ergodic theorem)**.**

$$\frac{1}{N}\sum_{t=1}^{N} f(X_t) \to \mathbb{E}_\pi[f(X)] \quad a.s.$$

2.19.3 Reversibility

Definition 2.19.7. π satisfies detailed balance if

$$\pi(x)K(x,y) = \pi(y)K(y,x).$$

Proposition 2.19.1. *Detailed balance* $\Rightarrow$ *stationarity.*

Remark 2.19.3. Reversibility makes it easy to design Markov chains.

2.20 Metropolis–Hastings

Target density: $p(x) \propto \tilde{p}(x)$.

Theorem 2.20.1. *The MH chain has stationary distribution* p.

Gibbs Sampling

Definition 2.20.1. At each step:

$$X_i \sim p(x_i | x_{-i}).$$

Algorithm 4 Metropolis–Hastings

Input: $\tilde{p}(x)$, proposal $q(x, y)$
for $t = 0, \ldots, T$ **do**
Sample proposal $\widehat{X}_{t+1} \sim q(X_t, \cdot)$
Accept with probability

$$\alpha(x, y) = \min \left\{ \frac{\tilde{p}(y)q(y, x)}{\tilde{p}(x)q(x, y)}, 1 \right\}$$

end for

Proposition 2.20.1. *Gibbs sampling is a special case of Metropolis–Hastings with acceptance probability* 1*.*

Remark 2.20.1. In graphical models:

$$p(x_i|x_{-i}) = p(x_i|x_{\mathrm{MB}(i)}).$$

Examples

Example 2.20.1 (Ball walk)**.**

$$\widehat{X}_{t+1} \sim \mathrm{Uniform}(B_R(X_t)),$$

then accept/reject via MH.

Example 2.20.2 (Gibbs sampling)**.** Select coordinate i and sample

$$X_i \sim p(x_i|x_{-i}).$$

Takeaways

- Direct sampling is rarely possible
- MCMC constructs a Markov chain with target stationary distribution
- Convergence requires irreducibility + aperiodicity
- Reversibility simplifies construction (MH)
- Gibbs exploits conditional structure

2.21 Markov Chain Monte Carlo (MCMC)

2.21.1 Sampling via Markov Chains

Definition 2.21.1. Goal: sample from a distribution

$$p(x) \propto e^{f(x)}, \quad x \in \Omega,$$

within ε in total variation distance.

Remark 2.21.1. We construct a Markov chain whose stationary distribution is p, then simulate it.

Rejection Sampling (Limitations)

Theorem 2.21.1. *If $p(x) \leq cq(x)$, rejection sampling produces samples from p, and*

$$\#trials \sim \mathrm{Geom}(1/c).$$

Remark 2.21.2. In high dimensions, c typically grows exponentially, making rejection sampling impractical.

Markov Chains

Definition 2.21.2. A sequence (X_t) is a Markov chain if

$$p(x_{t+1}|x_1, \ldots, x_t) = p(x_{t+1}|x_t).$$

Definition 2.21.3. A distribution π is stationary if

$$\pi K = \pi.$$

2.21.2 Reversibility

Definition 2.21.4. π satisfies detailed balance if

$$\pi(x)K(x, y) = \pi(y)K(y, x).$$

Proposition 2.21.1. *If detailed balance holds, then π is stationary.*

Proof.

$$\sum_x \pi(x)K(x, y) = \sum_x \pi(y)K(y, x) = \pi(y) \sum_x K(y, x) = \pi(y).$$

□

Metropolis–Hastings

Theorem 2.21.2. *The Metropolis–Hastings chain has stationary distribution p.*

Proof. Check detailed balance:

$$p(x)K(x, y) = p(x)q(x, y)\alpha(x, y) = \min\{p(x)q(x, y), p(y)q(y, x)\},$$

which is symmetric in (x, y). □

Algorithm 5 Metropolis–Hastings

Input: $\tilde{p}(x)$, proposal $q(x, y)$
for $t = 0, \ldots, T$ **do**
  Sample $Y \sim q(X_t, \cdot)$
  Accept with probability

$$\alpha(x, y) = \min\left\{\frac{\tilde{p}(y)q(y, x)}{\tilde{p}(x)q(x, y)}, 1\right\}$$

  Set $X_{t+1} = Y$ if accepted, else $X_{t+1} = X_t$
end for

Gibbs Sampling

Definition 2.21.5. At each step, pick coordinate i and sample

$$X_i \sim p(x_i | x_{-i}).$$

Proposition 2.21.2. *Gibbs sampling is a special case of Metropolis–Hastings with acceptance probability* 1*.*

Remark 2.21.3. In graphical models:

$$p(x_i | x_{-i}) = p(x_i | x_{\mathrm{MB}(i)}).$$

2.22 Langevin Monte Carlo (LMC)

Continuous-time process

Definition 2.22.1. Langevin diffusion:

$$dX_t = -\nabla V(X_t)\, dt + \sqrt{2}\, dB_t.$$

Remark 2.22.1. Target distribution:

$$p(x) \propto e^{-V(x)}.$$

Discretization

Using Euler–Maruyama:

$$X_{t+1} = X_t - \eta \nabla V(X_t) + \sqrt{2\eta}\, \xi_t, \quad \xi_t \sim \mathcal{N}(0, I).$$

Remark 2.22.2. This is noisy gradient descent.

Stationary Distribution

Theorem 2.22.1. *The Langevin diffusion has stationary distribution*

$$p(x) \propto e^{-V(x)}.$$

Proof. From the Fokker–Planck equation:

$$\frac{\partial \mu_t}{\partial t} = \nabla \cdot (\nabla V \, \mu_t) + \Delta \mu_t.$$

Plug $\mu(x) = e^{-V(x)}$:

$$\nabla \mu = -(\nabla V)e^{-V}, \quad \Delta \mu = \left(-\Delta V + \|\nabla V\|^2\right) e^{-V}.$$

Then:

$$\nabla \cdot (\nabla V \mu) = (\Delta V)e^{-V} - \|\nabla V\|^2 e^{-V}.$$

Adding:

$$\nabla \cdot (\nabla V \mu) + \Delta \mu = 0.$$

Thus μ is stationary. □

2.22.1 MALA (Metropolis-adjusted Langevin)

Algorithm 6 MALA

Propose

$$Y \sim \mathcal{N}(x - h\nabla V(x), 2hI)$$

Accept with MH correction

Remark 2.22.3. Corrects discretization bias of LMC.

2.23 Underdamped Langevin Dynamics

Definition 2.23.1. Introduce momentum v:

$$\begin{aligned} dX_t &= v_t \, dt \\ dv_t &= -\gamma v_t \, dt - \nabla V(X_t) \, dt + \sqrt{2\gamma} \, dB_t \end{aligned}$$

Remark 2.23.1. Stationary distribution:

$$p(x, v) \propto e^{-V(x) - \frac{1}{2}\|v\|^2}.$$

Remark 2.23.2. Non-reversible → faster mixing.

Hamiltonian Monte Carlo (HMC)

Hamiltonian dynamics

Define:

$$H(q, p) = U(q) + K(p)$$

$$\frac{dq}{dt} = \nabla_p H = \nabla K(p)$$
$$\frac{dp}{dt} = -\nabla_q H = -\nabla U(q)$$

Proposition 2.23.1. *Hamiltonian dynamics preserve:*

- *Energy:* $H(q(t), p(t))$
- *Volume (Liouville's theorem)*

Reversibility trick

Define momentum flip:

$$R(q, p) = (q, -p).$$

Then:

$$R \circ S_T \circ R = S_T^{-1}.$$

Proposition 2.23.2. *HMC transition is reversible w.r.t.*

$$\mu(q, p) \propto e^{-H(q,p)}.$$

Algorithm

Algorithm 7 Hamiltonian Monte Carlo (idealized)

Sample $p \sim e^{-K(p)}$
Simulate Hamiltonian dynamics for time T
Output q

Remark 2.23.3. No rejection needed in exact dynamics (but required after discretization).

Leapfrog Integrator

Remark 2.23.4. Properties:

- volume-preserving
- reversible
- approximately energy-preserving

Algorithm 8 Leapfrog

$p \leftarrow p - \frac{h}{2}\nabla U(q)$
for $t = 1, \dots, T$ **do**
  $q \leftarrow q + h\nabla K(p)$
  $p \leftarrow p - h\nabla U(q)$
end for
$p \leftarrow p - \frac{h}{2}\nabla U(q)$

Takeaways

- MCMC constructs Markov chains with target stationary distribution
- Metropolis–Hastings ensures correctness via detailed balance
- Gibbs exploits conditional structure
- Langevin adds gradient information (diffusion-based sampling)
- HMC uses Hamiltonian dynamics for efficient exploration

Langevin Monte Carlo

We want to sample from

$$p(x) \propto e^{-V(x)}.$$

Definition 2.23.2. The Langevin diffusion is the stochastic differential equation

$$dX_t = -\nabla V(X_t)\, dt + \sqrt{2}\, dB_t.$$

Discretizing via Euler–Maruyama:

$$X_{t+1} = X_t - \eta\nabla V(X_t) + \sqrt{2\eta}\, \xi_t, \quad \xi_t \sim \mathcal{N}(0, I_d).$$

Remark 2.23.5. This can be viewed as gradient descent with injected Gaussian noise.

Remark 2.23.6. The continuous-time process has stationary distribution $p(x) \propto e^{-V(x)}$, but the discretized version introduces bias unless η is small.

Metropolis-Adjusted Langevin Algorithm (MALA)

To correct discretization bias, use Langevin as a proposal in Metropolis–Hastings.

Definition 2.23.3. Proposal distribution:

$$q(x, y) \propto \exp\left(-\frac{\|y - (x - \eta\nabla V(x))\|^2}{4\eta}\right).$$

Algorithm 9 MALA

Propose $Y \sim \mathcal{N}(x - \eta \nabla V(x),\, 2\eta I)$
Accept with probability

$$\alpha(x, y) = \min\left\{\frac{p(y)q(y, x)}{p(x)q(x, y)}, 1\right\}$$

Underdamped Langevin Dynamics

Definition 2.23.4. Introduce momentum v:

$$\begin{aligned} dX_t &= v_t\, dt, \\ dv_t &= -\gamma v_t\, dt - \nabla V(X_t)\, dt + \sqrt{2\gamma}\, dB_t. \end{aligned}$$

Remark 2.23.7. Stationary distribution:

$$p(x, v) \propto e^{-V(x) - \frac{1}{2}\|v\|^2}.$$

Remark 2.23.8. Noise is injected only in v_t, leading to more efficient exploration.

Continuous-Time Markov Processes

Definition 2.23.5. A continuous-time Markov process $(X_t)_{t \geq 0}$ satisfies

$$P(X_u \in A | X_{[0,t]}) = P(X_u \in A | X_t), \quad u > t.$$

Definition 2.23.6. A family $(P_t)_{t \geq 0}$ is a Markov semigroup if

$$P_{t+s} = P_t P_s, \quad P_0 = \text{id}.$$

Generator of a Markov Process

Definition 2.23.7. The generator $\mathcal{L}$ is

$$\mathcal{L} f = \left| \frac{d}{dt} P_t f \right|_{t=0}.$$

Remark 2.23.9. This defines the process via

$$P_t = e^{t\mathcal{L}}.$$

2.23.1 Fokker–Planck Equation

Let $\mathcal{L}^*$ be the adjoint operator.

Definition 2.23.8. The distribution μ_t evolves as

$$\frac{d\mu_t}{dt} = \mathcal{L}^* \mu_t.$$

Remark 2.23.10. For Langevin diffusion:

$$\mathcal{L}f = -\langle \nabla f, \nabla V \rangle + \Delta f,$$

$$\mathcal{L}^* \mu = \nabla \cdot (\nabla V \mu) + \Delta \mu.$$

Stationarity of Langevin

Theorem 2.23.1. *The distribution*

$$\mu(x) \propto e^{-V(x)}$$

is stationary for Langevin diffusion.

Proof. Plug into Fokker–Planck equation:

$$\mathcal{L}^* \mu = \nabla \cdot (\nabla V \mu) + \Delta \mu.$$

Compute:

$$\nabla \mu = -(\nabla V)\mu, \quad \Delta \mu = (-\Delta V + \|\nabla V\|^2)\mu.$$

Then:

$$\nabla \cdot (\nabla V \mu) = (\Delta V)\mu - \|\nabla V\|^2 \mu.$$

Adding:

$$(\Delta V - \|\nabla V\|^2)\mu + (-\Delta V + \|\nabla V\|^2)\mu = 0.$$

□

2.24 Hamiltonian Monte Carlo (HMC)

2.24.1 Hamiltonian Dynamics

Define

$$H(q, p) = U(q) + K(p).$$

Dynamics:

$$\frac{dq}{dt} = \nabla_p H, \quad \frac{dp}{dt} = -\nabla_q H.$$

Remark 2.24.1. Typical choice:

$$K(p) = \frac{1}{2}\|p\|^2.$$

2.24.2 Key Properties

Proposition 2.24.1. *Hamiltonian dynamics preserve:*

- *energy* $H(q, p)$
- *volume in phase space*

2.24.3 Leapfrog Integrator

Lemma 2.24.1. *The leapfrog integrator:*

- *preserves volume*
- *is reversible (up to momentum flip)*

Remark 2.24.2. It does *not* exactly preserve the Hamiltonian.

HMC with Leapfrog

Algorithm 10 Hamiltonian Monte Carlo (practical)

Sample $p \sim e^{-K(p)}$
Simulate Hamiltonian dynamics via leapfrog for time T
Accept with probability

$$\alpha = \min\{e^{-H(q',p')+H(q,p)}, 1\}$$

Remark 2.24.3. The Metropolis step corrects discretization error.

Choosing Parameters

- Integration time T:
  - Too small $\Rightarrow$ slow exploration
  - Too large $\Rightarrow$ wasted computation
- Randomizing T helps avoid periodic behavior
- No-U-Turn criterion adaptively selects trajectory length

Geometric Adaptation

Definition 2.24.1. Riemannian HMC uses position-dependent metric:

$$K(q, p) = \frac{1}{2}p^\top \Sigma^{-1}(q)p + \frac{1}{2}\log\det\Sigma(q).$$

2.25 Convergence of Markov Chains

2.25.1 Spectral Gap

Theorem 2.25.1. *Let λ_2 be the second largest eigenvalue. Then*

$$\mathrm{Var}_\mu(K^n f) \leq |\lambda_2|^{2n} \mathrm{Var}_\mu(f).$$

2.25.2 Non-reversible Chains

Remark 2.25.1. For non-reversible chains:

$$\chi^2(\nu K^n || \mu) \leq C|\lambda_2|^{2n} \chi^2(\nu || \mu).$$

Remark 2.25.2. In continuous time this is called *hypocoercivity*.

2.25.2.1 Poincaré Inequality

Definition 2.25.1. A distribution μ satisfies a Poincaré inequality if

$$\langle g, \mathcal{L} g \rangle_\mu \geq c \mathrm{Var}_\mu(g).$$

Remark 2.25.3. Implies exponential convergence rate e^{-2ct}.

2.25.3 Contraction in Divergences

Definition 2.25.2. A kernel K is contractive if

$$D_f(\nu K || \mu) \leq (1-\kappa) D_f(\nu || \mu).$$

Remark 2.25.4. Implies geometric convergence.

2.25.4 Mixing Time

Definition 2.25.3.

$$t_{\mathrm{mix}}(\varepsilon) = \min\{t : \sup_{\mu_0} \mathrm{TV}(\mu_0 K^t, \mu) \leq \varepsilon\}.$$

Definition 2.25.4. Relaxation time:

$$t_{\mathrm{rel}} = \frac{1}{1 - |\lambda_2|}.$$

2.25.5 Conductance

Definition 2.25.5. For $S \subset \Omega$:

$$\Phi(S) = \frac{Q(S,S^c)}{\mu(S)}, \quad Q(S,S^c) = \sum_{x \in S, y \in S^c} \mu(x) P(x,y).$$

Definition 2.25.6.

$$\Phi_* = \min_{\mu(S) \leq 1/2} \Phi(S).$$

Theorem 2.25.2.

$$t_{\text{mix}}(1/4) \geq \frac{1}{4\Phi_*}.$$

2.25.6 Asymptotic Variance

Definition 2.25.7.

$$\sigma_f^2 = \text{Var}_\mu(f(X_0)) + 2 \sum_{k=1}^{\infty} \text{Cov}_\mu(f(X_0), f(X_k)).$$

Theorem 2.25.3.

$$\frac{1}{N} \sum_{k=0}^{N-1} f(X_k) \to \mathbb{E}_\mu[f]$$

and

$$\text{Var}\left(\frac{1}{N} \sum f(X_k)\right) \sim \frac{\sigma_f^2}{N}.$$

2.25.7 Practical Diagnostics

- Trace plots (check stationarity)
- Autocorrelation:
$$\rho_\ell = \frac{\text{Cov}(f(X_0), f(X_\ell))}{\text{Var}(f)}$$
- Effective sample size:
$$N_{\text{eff}} = \frac{N}{1 + 2\sum_{\ell \geq 1} \rho_\ell}$$

Discussion

- Single long chain vs multiple shorter chains:
  - single chain: cheaper burn-in
  - multiple chains: better exploration of multimodal distributions

2.26 Tempering and Particle Methods

2.26.1 Multimodality and Slow Mixing

Multimodal target distributions pose a fundamental challenge for MCMC.

- Probability mass is concentrated in separated regions (modes).
- Transitions between modes require crossing *energy barriers*.
- Local proposals (e.g., random walk, Langevin) rarely cross these barriers.

Remark 2.26.1. For distributions of the form $p(x) \propto e^{-f(x)}$, the time to transition between modes is often *exponential* in the barrier height:

$$\text{mixing time} \sim \exp(\Delta f).$$

This is known as *metastability*.

2.26.2 Temperature Scaling

Introduce a *temperature parameter* $T > 0$:

$$p_T(x) \propto e^{-f(x)/T} = e^{-\beta f(x)}, \quad \beta = \frac{1}{T}.$$

- High temperature ($T \gg 1$, $\beta \ll 1$): distribution is *flattened*.
- Low temperature ($T \ll 1$, $\beta \gg 1$): distribution concentrates near modes.

Remark 2.26.2. In Langevin dynamics:

$$dX_t = -\nabla V(X_t)\,dt + \sqrt{2T}\,dB_t,$$

temperature controls the tradeoff between gradient (drift) and noise.

2.26.3 Simulated Annealing

Simulated annealing gradually lowers temperature:

$$T_1 > T_2 > \cdots \to 0.$$

- Initially explores broadly.
- Eventually concentrates near global minima.

Remark 2.26.3. Once modes separate at low temperature, the chain becomes *trapped*: probability mass cannot easily move between modes.

2.26.4 Simulated Tempering

Introduce temperature as an auxiliary variable:

$$(\ell, x) \in \{1, \ldots, L\} \times \Omega.$$

Define target:

$$\pi(\ell, x) \propto w_\ell \, p_\ell(x), \quad p_\ell(x) \propto e^{-f(x)/T_\ell}.$$

Algorithm:

- Alternate:
  - Update x using MCMC at fixed temperature ℓ.
  - Propose $\ell \to \ell \pm 1$ with MH acceptance:

$$\alpha = \min \left\{ \frac{w_{\ell'} p_{\ell'}(x)}{w_\ell p_\ell(x)}, 1 \right\}.$$

Remark 2.26.4. Drawback: requires knowledge (or estimation) of normalizing constants for p_ℓ.

2.26.5 Parallel Tempering (Replica Exchange)

Run multiple chains at different temperatures:

$$(x_1, \ldots, x_L), \quad x_\ell \sim p_\ell.$$

Target:

$$\pi(x_1, \ldots, x_L) = \prod_{\ell=1}^{L} p_\ell(x_\ell).$$

Swap proposal:

$$(x_i, x_{i+1}) \to (x_{i+1}, x_i),$$

with acceptance probability

$$\alpha = \min \left\{ \frac{p_i(x_{i+1}) p_{i+1}(x_i)}{p_i(x_i) p_{i+1}(x_{i+1})}, 1 \right\}.$$

Remark 2.26.5. Advantages:

- No need for normalizing constants.
- High-temperature chains help low-temperature chains escape modes.

2.26.6 Sequential Monte Carlo (SMC)

Construct a sequence of distributions:

$$p_k(x) \propto \tilde{p}_k(x), \quad k = 1, \dots, L,$$

typically interpolating between easy and target distributions.

Algorithm 11 Sequential Monte Carlo

1: Sample $X_1^{(i)} \sim p_1$
2: **for** $k = 2$ to L **do**
3: Compute weights:

$$w_k^{(i)} \propto \frac{\tilde{p}_k(X_{k-1}^{(i)})}{\tilde{p}_{k-1}(X_{k-1}^{(i)})}$$

4: Resample particles according to $w_k^{(i)}$
5: Move particles: $X_k^{(i)} \sim K_k(X_{k-1}^{(i)}, \cdot)$
6: **end for**
7: Output $\{X_L^{(i)}\}$

Remark 2.26.6. SMC combines:

- Importance sampling (weights)
- Resampling (to prevent degeneracy)
- MCMC moves (to diversify particles)

2.26.7 Annealed Importance Sampling (AIS)

A simplified version of SMC without resampling.

Algorithm 12 Annealed Importance Sampling

1: Sample $X_1^{(i)} \sim p_1$
2: **for** $k = 2$ to L **do**
3: Sample $X_k^{(i)} \sim K_k(X_{k-1}^{(i)}, \cdot)$
4: **end for**
5: Compute weights:

$$\tilde{w}^{(i)} = \prod_{k=2}^{L} \frac{\tilde{p}_k(X_k^{(i)})}{\tilde{p}_{k-1}(X_k^{(i)})}$$

6: Estimate:

$$\mathbb{E}_{p_L}[f(X)] \approx \frac{\sum_i \tilde{w}^{(i)} f(X_L^{(i)})}{\sum_i \tilde{w}^{(i)}}$$

Remark 2.26.7. AIS is especially useful for:

- Estimating partition functions
- Bridging between simple and complex distributions

Key Insight

Tempering and particle methods address the same core issue:

Exploration vs. concentration tradeoff

- High temperature $\Rightarrow$ exploration
- Low temperature $\Rightarrow$ accurate sampling
- Tempering methods: move between temperatures
- Particle methods: propagate distributions across temperatures

2.27 Variational Methods

2.27.1 MCMC vs Variational Inference

Two paradigms for approximate inference:

- **MCMC:**
  - Constructs a Markov chain with stationary distribution p
  - Asymptotically exact
  - Slow mixing; hard to diagnose convergence
- **Variational methods:**
  - Solve an optimization problem over distributions
  - Fast and scalable
  - Approximate; may converge to local optima

Remark 2.27.1. MCMC: accuracy $\uparrow$, speed $\downarrow$
Variational inference: speed $\uparrow$, accuracy $\downarrow$ (approximation bias)

2.27.2 Gibbs Variational Principle

Let

$$p(x) = \frac{e^{f(x)}}{Z}, \quad Z = \int e^{f(x)} dx.$$

Lemma 2.27.1 (Gibbs variational principle)**.**

$$\ln Z = \max_q \left\{ H(q) + \mathbb{E}_{x\sim q}[f(x)] \right\}.$$

The maximum is attained at $q = p$.

Proof. Compute:

$$\mathrm{KL}(q\|p) = \int q(x) \log \frac{q(x)}{p(x)} dx = \int q(x) \log q(x) dx - \int q(x) f(x) dx + \log Z.$$

Rearrange:

$$\log Z = H(q) + \mathbb{E}_q[f(x)] + \mathrm{KL}(q\|p).$$

Since $\mathrm{KL}(q\|p) \geq 0$,

$$\log Z \geq H(q) + \mathbb{E}_q[f(x)],$$

with equality iff $q = p$. □

2.27.3 Variational Relaxation

We cannot optimize over all distributions q. Restrict to a class $\mathcal{Q}$:

$$\log Z \geq \max_{q\in\mathcal{Q}} \left\{ H(q) + \mathbb{E}_q[f(x)] \right\}.$$

Remark 2.27.2. Tradeoff:

- Larger $\mathcal{Q}$: better approximation, harder optimization
- Smaller $\mathcal{Q}$: easier optimization, worse approximation

2.27.4 KL Projection Interpretation

Using the identity:

$$\log Z = \mathrm{KL}(q\|p) + H(q) + \mathbb{E}_q[f(x)],$$

we obtain:

$$\arg\min_{q\in\mathcal{Q}} \mathrm{KL}(q\|p) = \arg\max_{q\in\mathcal{Q}} \left\{ H(q) + \mathbb{E}_q[f(x)] \right\}.$$

Remark 2.27.3. Variational inference = **projection of p onto $\mathcal{Q}$ in KL divergence**.

2.27.5 Two Use Cases

Partition function approximation:

$$\log Z \geq \max_{q\in\mathcal{Q}} \left\{ H(q) + \mathbb{E}_q[f(x)] \right\}.$$

Distribution approximation:

$$q^* = \arg\min_{q\in\mathcal{Q}} \mathrm{KL}(q\|p).$$

For latent-variable models:

$$p(x) = \int p(x,z)dz,$$

we obtain:

$$\log p(x) = \mathrm{KL}(q(z)\|p(z|x)) + H(q) + \mathbb{E}_q[\log p(x,z)].$$

2.27.6 Evidence Lower Bound (ELBO)

Definition 2.27.1.

$$\mathrm{ELBO}(q) = H(q) + \mathbb{E}_{z\sim q}[\log p(x,z)].$$

$$\log p(x) \geq \mathrm{ELBO}(q).$$

Remark 2.27.4. Maximizing ELBO is equivalent to minimizing:

$$\mathrm{KL}(q(z)\|p(z|x)).$$

2.27.7 Mean-Field Variational Inference

Assume factorization:

$$q(x_1,\ldots,x_n) = \prod_{i=1}^{n} q_i(x_i).$$

Optimize coordinate-wise:

$$q_i^*(x_i) \propto \exp\left(\mathbb{E}_{q_{-i}}[f(x)]\right).$$

Algorithm 13 Coordinate Ascent Variational Inference (CAVI)

1: Initialize $q_1, \ldots, q_n$
2: **repeat**
3:     **for** $i = 1, \ldots, n$ **do**
4:
$$q_i(x_i) \propto \exp\left(\mathbb{E}_{q_{-i}}[f(x)]\right)$$
5:     **end for**
6: **until** convergence

2.27.8 Approximate Likelihood-Based Learning

For latent-variable models $p_\theta(x, z)$:

$$\log p_\theta(x) = \max_q \{H(q) + \mathbb{E}_q[\log p_\theta(x, z)]\}.$$

Instead optimize:

$$\max_\theta \max_q \sum_{i=1}^{N} [H(q_i) + \mathbb{E}_{z\sim q_i} \log p_\theta(x_i, z)].$$

2.27.9 Amortized Variational Inference

Parameterize:

$$q_\phi(z|x).$$

Objective:

$$\max_{\theta,\phi} \sum_{i=1}^{N} \mathbb{E}_{z\sim q_\phi(z|x_i)} \left[\log \frac{p_\theta(x_i, z)}{q_\phi(z|x_i)}\right].$$

2.27.10 Variational Autoencoder (VAE)

- Encoder: $q_\phi(z|x)$
- Decoder: $p_\theta(x|z)$

ELBO:

$$\mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}(q_\phi(z|x)\|p(z)).$$

Remark 2.27.5.
- First term: reconstruction
- Second term: regularization toward prior

2.27.11 Gradient Estimators

Score-function (REINFORCE):

$$\nabla_\phi \mathbb{E}_{z\sim q_\phi}[f(z)] = \mathbb{E}_{z\sim q_\phi}[f(z)\nabla_\phi \log q_\phi(z)].$$

Reparameterization trick:

$$z = g(x, \xi, \phi), \quad \xi \sim p(\xi),$$
$$\nabla_\phi \mathbb{E}_{z\sim q_\phi}[f(z)] = \mathbb{E}_\xi[\nabla_\phi f(g(x, \xi, \phi))].$$

Example 2.27.1.

$$z \sim \mathcal{N}(\mu_\phi(x), \Sigma_\phi(x)) \quad \Rightarrow \quad z = \mu_\phi(x) + \Sigma_\phi^{1/2}(x)\,\xi.$$

Remark 2.27.6.
- REINFORCE: general but high variance
- Reparameterization: lower variance but requires differentiability

2.27.12 Expectation Maximization (EM)

Algorithm 14 Expectation Maximization

1: **repeat**
2: **E-step:** $q(z) \leftarrow p_\theta(z|x)$
3: **M-step:** $\theta \leftarrow \arg\max_\theta \mathbb{E}_q[\log p_\theta(x, z)]$
4: **until** convergence

Remark 2.27.7. EM is a special case of variational inference with $\mathcal{Q}$ equal to all distributions.

2.27.13 Key Insight

Variational inference replaces:

$$\text{sampling} \quad \longrightarrow \quad \text{optimization.}$$

- MCMC: asymptotically exact, expensive
- Variational: approximate, scalable

2.27.14 Expectation Maximization (EM)

We consider latent-variable models $p_\theta(x, z)$ with observed data $x_1, \ldots, x_n$.

$$\max_\theta \sum_{i=1}^{n} \log p_\theta(x_i) = \max_\theta \sum_{i=1}^{n} \log \int p_\theta(x_i, z) dz.$$

Using the variational bound,

$$\log p_\theta(x_i) \geq \max_{q_i} \left[H(q_i) + \mathbb{E}_{z\sim q_i} \log p_\theta(x_i, z) \right].$$

Algorithm 15 Expectation Maximization (EM)

1: Initialize θ
2: **repeat**
3: **E-step:**
$$q_i(z) \leftarrow p_\theta(z|x_i)$$
4: **M-step:**
$$\theta \leftarrow \arg\max_\theta \sum_{i=1}^{n} \mathbb{E}_{z\sim q_i}[\log p_\theta(x_i, z)]$$
5: **until** convergence

Remark 2.27.8. EM alternates between:

- computing posteriors over latent variables
- maximizing expected complete-data log-likelihood

2.27.15 Example: Mixture of Gaussians

Model:

$$Z_m \sim \text{Categorical}(\pi), \quad X_m \sim \mathcal{N}(\mu_{Z_m}, \Sigma_{Z_m}).$$

E-step: posterior responsibilities

$$z_{mj} := p(Z_m = j | X_m = x_m) = \frac{\pi_j \, \mathcal{N}(x_m; \mu_j, \Sigma_j)}{\sum_k \pi_k \, \mathcal{N}(x_m; \mu_k, \Sigma_k)}.$$

M-step:

$$\pi_j \leftarrow \frac{1}{n} \sum_{m=1}^{n} z_{mj},$$

$$\mu_j \leftarrow \frac{\sum_{m=1}^{n} z_{mj} x_m}{\sum_{m=1}^{n} z_{mj}},$$

$$\Sigma_j \leftarrow \frac{\sum_{m=1}^{n} z_{mj} (x_m - \mu_j)(x_m - \mu_j)^\top}{\sum_{m=1}^{n} z_{mj}}.$$

2.27.16 Outer Relaxation

Instead of optimizing over global distributions $q(x)$, optimize over local marginals.

Let $p(x) \propto \exp\left(\sum_C \phi_C(x_C)\right)$.

Define pseudo-marginals $\{q_C\}$:

$$\mathbb{E}_q \left[\sum_C \phi_C(x_C) \right] = \sum_C \mathbb{E}_{q_C}[\phi_C(x_C)].$$

Relaxation:

$$\log Z \approx \max_{\{q_C\} \in \mathcal{Q}} \left[H(q) + \sum_C \mathbb{E}_{q_C}[\phi_C(x_C)] \right].$$

Remark 2.27.9. Constraints:

- consistency between marginals
- $q_C(x_C) \geq 0$, $\sum q_C = 1$

2.27.17 Marginal Polytope vs Local Polytope

- **Marginal polytope:** exact set of realizable marginals (intractable)
- **Local polytope:** enforce only pairwise consistency

Remark 2.27.10. Membership in marginal polytope is NP-hard.

2.27.18 Tree Case (Exact)

Lemma 2.27.2. *If the factor graph is a tree, local consistency implies global consistency.*

$$q(x) = \frac{\prod_{a\in F} q_a(x_a)}{\prod_{i\in V} q_i(x_i)^{d_i-1}}.$$

2.27.19 Bethe Entropy

Lemma 2.27.3. *For tree-structured graphs,*

$$H(q) = \sum_{a\in F} H(q_a) - \sum_{i\in V} (d_i - 1) H(q_i).$$

Definition 2.27.2. The RHS defines the **Bethe entropy**:

$$H_{\text{Bethe}}(\{q_a\}) := \sum_a H(q_a) - \sum_i (d_i - 1) H(q_i).$$

2.27.20 Bethe Free Energy

Optimization problem:

$$\max_{\{q_a\}\in\mathcal{Q}} \left[H_{\text{Bethe}}(\{q_a\}) + \sum_a \mathbb{E}_{q_a}[\phi_a(x_a)] \right].$$

Remark 2.27.11.

- Exact on trees
- Approximate on loopy graphs
- Not necessarily convex

Note. Stationary points correspond to fixed points of loopy belief propagation.

2.27.21 KL Projections

Two types:

- **I-projection:**

  $$\min_{q\in\mathcal{Q}} \mathrm{KL}(q\|p)$$

  underestimates variance

- **M-projection:**
$$\min_{q\in\mathcal{Q}} \mathrm{KL}(p\|q)$$
matches moments

2.27.22 M-Projection for Exponential Families

Let
$$q_\theta(x) = \frac{\exp(\langle\theta, \phi(x)\rangle)}{Z(\theta)}.$$

Lemma 2.27.4 (Moment matching)**.**
$$\arg\min_\theta \mathrm{KL}(p\|q_\theta) = \theta^* \quad s.t. \quad \mathbb{E}_{q_{\theta^*}}[\phi(x)] = \mathbb{E}_p[\phi(x)].$$

2.27.23 Belief Propagation as Variational Inference

Messages correspond to approximate marginals:
$$\nu_{i\to a}(x_i) \approx \text{marginal of } x_i.$$

Updates correspond to projections onto exponential family approximations.

Remark 2.27.12. Variational view:

- BP = coordinate ascent on Bethe free energy
- Loopy BP = approximate inference via Bethe relaxation

2.27.24 Example: Bayesian Linear Model

$$y = Ax + w, \quad w \sim \mathcal{N}(0, \sigma^2 I).$$

Posterior:
$$p(x|y) \propto \exp\left(-\frac{1}{2\sigma^2}\|y - Ax\|^2\right) p_0(x).$$

If
$$p_0(x) \propto e^{-\beta\|x\|_1},$$
then MAP becomes:
$$\min_x \|y - Ax\|^2 + \lambda\|x\|_1,$$
i.e., LASSO.

2.27.25 Approximate Message Passing (AMP)

To obtain tractable updates:

- Assume large random matrix A
- Use Gaussian approximations for messages

Remark 2.27.13. AMP emerges as a simplification of belief propagation under high-dimensional limits.

Chapter 3

Modern Deep Generative Models

3.1 Overview of Deep Generative Modeling

Goal: learn a distribution $p_\theta(x)$ from samples $\{x_i\}_{i=1}^N$ such that we can generate new samples.

Two broad paradigms:

- **Likelihood-based methods:**
  - Energy-based models (EBM)
  - Variational autoencoders (VAE)
  - Normalizing flows
- **Non-likelihood-based methods:**
  - Generative adversarial networks (GAN)
  - Noise contrastive estimation (NCE)
  - Score-based / diffusion models

Remark 3.1.1. Likelihood-based methods require evaluating or approximating $p_\theta(x)$, often needing inference (MCMC or variational).

3.2 Generative Modeling via Transformations

Let $z \sim p_0(z)$ be a simple distribution (e.g., $\mathcal{N}(0, I)$), and define

$$x = g_\theta(z).$$

Then p_θ is the pushforward:

$$p_\theta = (g_\theta)_\# p_0.$$

If g_θ is invertible:

$$p_\theta(x) = p_0(g_\theta^{-1}(x)) \left|\det Dg_\theta^{-1}(x)\right|.$$

Remark 3.2.1. This is the foundation of **normalizing flows**.

3.3 Normalizing Flows

3.3.1 Definition

Let $z \sim p_0(z)$ and $x = g(z)$ with invertible g. Then:

$$\log p(x) = \log p_0(z) + \log \left|\det Dg^{-1}(x)\right|.$$

Remark 3.3.1. Key requirement: efficient computation of $\det Dg^{-1}(x)$.

3.3.2 Composition of Flows

Let

$$g = g_L \circ \cdots \circ g_1.$$

Then:

$$\log \left|\det Dg^{-1}(x)\right| = \sum_{\ell=1}^{L} \log \left|\det Df_\ell(x_\ell)\right|,$$

where $f_\ell = g_\ell^{-1}$.

Remark 3.3.2. Design principle: compose simple invertible transformations with tractable Jacobians.

3.4 Building Blocks for Flows

3.4.1 Affine Transformations

$$f(x) = Ax + b.$$

- $\det Df = \det A$
- Efficient if A is triangular
- Limited expressivity

3.4.2 Coordinate-wise Transformations

$$f(x) = (f_1(x_1), \ldots, f_d(x_d)).$$

$$\det Df = \prod_{i=1}^{d} f_i'(x_i).$$

Remark 3.4.1. Keeps coordinates independent $\Rightarrow$ limited modeling power.

3.5 Affine Coupling Layers

Partition variables:

$$x = (x_1, x_2).$$

Define:

$$z_1 = x_1, \quad z_2 = x_2 \odot \exp(s(x_1)) + t(x_1).$$

Jacobian is triangular:

$$\log|\det Df(x)| = \sum_i s_i(x_1).$$

Remark 3.5.1.

- Efficient determinant: $\mathcal{O}(d)$
- Used in NICE, RealNVP, Glow

3.6 Residual Flows

Lemma 3.6.1. *Let $f(x) = x + F(x)$ where F is L-Lipschitz with $L < 1$. Then f is bijective.*

Theorem 3.6.1 (Banach fixed point)**.** *If T is a contraction, then it has a unique fixed point.*

Remark 3.6.1. Residual flows ensure invertibility via contraction mappings.

3.7 Efficient Log-Determinant Computation

$$\log\det(I + DF) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} \operatorname{Tr}((DF)^k).$$

3.7.1 Hutchinson Trace Estimator

Lemma 3.7.1. *For $v \sim \mathcal{N}(0, I)$:*

$$\operatorname{Tr}(A) = \mathbb{E}[v^\top A v].$$

Remark 3.7.1. Allows stochastic estimation of log-determinants in high dimensions.

3.8 Flows in Variational Inference

Let

$$z = f_\phi(u, x), \quad u \sim \mathcal{N}(0, I).$$

Then ELBO becomes:

$$\mathbb{E}_u \left[\log p_\theta(x, f_\phi(u, x)) - \log q_\phi(f_\phi(u, x)|x)\right].$$

Remark 3.8.1. Flows increase expressivity of variational distributions.

3.9 Pros and Cons of Normalizing Flows

Pros:

- Exact likelihood evaluation
- Efficient sampling
- Invertible mapping enables interpolation

Cons:

- Requires invertibility constraints
- Jacobian computation can be costly
- May struggle with topological mismatch

3.10 Continuous-Time Flows (Neural ODEs)

Define dynamics:

$$\frac{dx(t)}{dt} = F(x(t), t), \quad x(0) = x_0.$$

Theorem 3.10.1 (Picard–Lindelöf)**.** *If F is Lipschitz, then the ODE has a unique solution.*

Remark 3.10.1. Defines a continuous flow:

$$x(T) = f_T(x_0).$$

3.11 Density Evolution

Change of variables:

$$\frac{d}{dt} \log p_t(x) = -\nabla \cdot F(x, t).$$

Remark 3.11.1. Leads to the **continuity equation**.

3.12 Score-Based Modeling

3.12.1 Score Function

Definition 3.12.1. The score function is:

$$s(x) = \nabla \log p(x).$$

3.12.2 Score Matching

$$L(\theta) = \mathbb{E}_{x\sim p}\|\nabla \log p(x) - s_\theta(x)\|^2.$$

Remark 3.12.1. Avoids computing normalization constant.

3.12.3 Integration by Parts Trick

$$\mathbb{E}\|\nabla \log p(x) - s_\theta(x)\|^2 = \mathbb{E}\left[\|s_\theta(x)\|^2 + \mathrm{Tr}(\nabla s_\theta(x))\right].$$

3.13 Denoising Score Matching

Add noise:

$$y = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

Proposition 3.13.1. *Optimal estimator satisfies:*

$$s(y) = \frac{1}{\sigma^2}(\mathbb{E}[x|y] - y).$$

Remark 3.13.1. Links score estimation to denoising.

3.14 Diffusion Models (Preview)

Forward process:

$$dx_t = \sqrt{2}dB_t.$$

Reverse process depends on score:

$$dx_t = \nabla \log p_t(x)dt + \sqrt{2}dB_t.$$

Remark 3.14.1. Learning the score allows sampling from p via reverse-time SDE.

3.15 Summary

- Normalizing flows: exact likelihood via invertible maps
- Neural ODEs: continuous-time flows
- Score-based models: learn $\nabla \log p$
- Diffusion: simulate reverse-time dynamics

Chapter 4

Modern Deep Generative Models (II): Diffusion and Score-Based Methods

4.1 Statistical Complexity of Score Matching

Definition 4.1.1. The **log-Sobolev constant** of a distribution Q on $\mathbb{R}^d$ is the smallest C_{LS} such that for all distributions P,

$$\mathrm{KL}(P\|Q) \leq C_{\text{LS}}\,\mathrm{FI}(P\|Q),$$

where FI is the Fisher divergence.

Remark 4.1.1.

- Stronger than Poincaré inequality
- Implies exponential convergence of Langevin diffusion:

$$\mathrm{KL}(P_t\|Q) \leq e^{-t/C_{\text{LS}}}\mathrm{KL}(P_0\|Q)$$

4.1.1 Statistical Efficiency of Score Matching

Theorem 4.1.1 (Forbes & Lauritzen, 2015)**.** *Let $X_1, \ldots, X_N \sim P_{\theta^*}$. Then the score matching estimator*

$$\hat{\theta}_{\text{SM}} = \arg\min_{\theta} \sum_{i=1}^{N} \left[\frac{1}{2}\|\nabla \log p_\theta(X_i)\|^2 + \mathrm{Tr}(\nabla^2 \log p_\theta(X_i))\right]$$

satisfies

$$\sqrt{N}(\hat{\theta}_{\text{SM}} - \theta^*) \xrightarrow{d} \mathcal{N}(0, \Gamma_{\text{SM}}).$$

Remark 4.1.2. Score matching is typically less statistically efficient than MLE.

Theorem 4.1.2 (Koehler et al., 2022)**.** *If P_{θ^*} satisfies a restricted Poincaré inequality, then*

$$\|\Gamma_{\text{SM}}\| \lesssim C_P^2\,\|I(\theta)\|^{-1} \cdot \textit{(moment terms)}.$$

Remark 4.1.3. Statistical gap depends on smoothness and functional inequalities.

4.2 Score Matching on Discrete Spaces

Definition 4.2.1. Let $G = (V, E)$ be a graph. Define the discrete gradient:

$$\nabla f(v) = (f(w) - f(v))_{w \in N(v)}.$$

Example (hypercube $\{\pm 1\}^n$):

$$\nabla_i f(x) = f(x^{(i)}) - f(x),$$

where $x^{(i)}$ flips coordinate i.

4.3 Pseudo-Likelihood

Definition 4.3.1. The pseudo-likelihood objective:

$$L_P(q) = \sum_{i=1}^{d} \mathbb{E}_{X \sim p}[\log q(X_i | X_{-i})].$$

Remark 4.3.1.

- Maximized at $q = p$
- Avoids global normalization
- Used in Ising models and graphical models

4.3.1 Generalized Pseudo-Likelihood

Partition coordinates into blocks $S_1, \ldots, S_k$:

$$L_P(q) = \sum_{i=1}^{k} \mathbb{E}_{X \sim p}[\log q(X_{S_i} | X_{S_i^c})].$$

4.4 Diffusion Models

4.4.1 Goal

Learn $p(x)$ and generate samples by transforming noise into data.

4.4.2 Forward Process

Define an SDE:

$$dx_t = f(x_t, t)dt + G(x_t, t)dW_t.$$

This gradually transforms data into noise.

4.4.3 Reverse Process

Theorem 4.4.1 (Anderson, 1982)**.** *The reverse-time SDE is:*

$$dy_t = \left[f(y_t, t) - \nabla \cdot (GG^\top p_t)(y_t)/p_t(y_t)\right] dt + G(y_t, t)d\widetilde{W}_t.$$

Remark 4.4.1. Reverse process depends on score $\nabla \log p_t(x)$.

4.4.4 Reverse Brownian Motion

$$dy_t = f(y_t, t)dt + G(y_t, t)d\widetilde{W}_t$$

is Brownian motion run backwards in time.

4.5 Score-Based Diffusion Framework

1. Define forward SDE:
$$dx_t = f(x_t, t)dt + g(t)dW_t$$
2. Learn score $s_\theta(x, t) \approx \nabla \log p_t(x)$
3. Simulate reverse SDE:

$$dx_t = \left[f(x_t, t) - g(t)^2 s_\theta(x_t, t)\right] dt + g(t)d\widetilde{W}_t$$

4.6 Variance Exploding (VE)

Forward process:

$$dx_t = g(t)dW_t$$

$$x_t = x_0 + \sigma(t)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Variance increases over time.

4.7 Variance Preserving (VP)

$$dx_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)}dW_t$$

Remark 4.7.1. Ornstein–Uhlenbeck process.

4.8 Discrete-Time Diffusion (DDPM)

Forward:

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\epsilon$$

Reverse:

$$p(x_{t-1}|x_t) \approx \mathcal{N}(\mu_\theta(x_t,t), \Sigma_t).$$

4.9 Connection to VAE

Remark 4.9.1. Diffusion models can be interpreted as hierarchical VAEs:

$$p(x_0, \ldots, x_T) = p(x_T) \prod_{t=1}^{T} p(x_{t-1}|x_t).$$

Training objective resembles ELBO:

$$\sum_t \mathbb{E}[\|\epsilon - \epsilon_\theta(x_t,t)\|^2].$$

4.10 Challenges with SDEs

- Brownian motion is not smooth
- Step size must be very small:
$$h = \mathcal{O}(1/d)$$

4.11 Probability Flow ODE

We can replace SDE with deterministic ODE:

$$dx_t = \left[f(x_t,t) - g(t)^2 \nabla \log p_t(x_t)\right] dt.$$

Remark 4.11.1.
- Same marginals as SDE
- Faster sampling (no noise)
- Sometimes less stable

4.12 Fokker–Planck Equation

Density evolution:

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot (f p_t) + \frac{1}{2}\nabla^2 (GG^\top p_t).$$

4.13 Summary of Diffusion Models

- Learn score function $\nabla \log p_t(x)$
- Simulate reverse-time process
- Two main variants:
  - Variance exploding (VE)
  - Variance preserving (VP / DDPM)
- Alternative: probability flow ODE

4.14 Score vs Likelihood-Based Learning

- Score matching:
  - avoids normalization constant
  - less statistically efficient
- MLE:
  - statistically optimal
  - computationally hard

Remark 4.14.1. Functional inequalities (Poincaré, log-Sobolev) govern both:

- mixing of Markov chains
- statistical efficiency of learning

4.15 Probability Flow ODE and Sampling Refinements

4.15.1 Probability Flow ODE

Definition 4.15.1. Given the forward SDE

$$dx_t = f(x_t, t)\, dt + g(t)\, dW_t,$$

the **probability flow ODE** is

$$dx_t = \Big[f(x_t, t) - g(t)^2 \nabla \log p_t(x_t)\Big]\, dt.$$

Remark 4.15.1.
- Shares the same marginal distributions as the SDE.
- Deterministic ⇒ faster sampling (no stochastic noise).

- Can compute likelihood via change-of-variables.
- May be less stable than SDE sampling.

4.15.2 Predictor–Corrector Methods

- **Predictor (P):** simulate reverse SDE/ODE step:

$$x_{t+h} \approx \text{reverse dynamics}$$

- **Corrector (C):** apply MCMC (Langevin step):

$$x \leftarrow x + h\nabla \log p_t(x) + \sqrt{2h}\,\xi, \quad \xi \sim \mathcal{N}(0, I).$$

- **PC scheme:** alternate P and C steps.

Remark 4.15.2. Corrector ensures local equilibrium; predictor moves across time.

4.16 Architecture and Implementation

4.16.1 Score Network

- Typically parameterized by a neural network $s_\theta(x, t)$
- For images: **U-Net architecture**
- Multi-scale structure critical for denoising

4.16.2 Practical Considerations

- Higher-order solvers (e.g., Heun)
- Noise schedule design
- Loss weighting across time
- Conditional diffusion models: learn $p(x|z)$
- Latent diffusion (VAE + diffusion)

4.17 Generalizing Diffusion Models

4.17.1 Observation Process View

Let $X \sim P$ be data. Define noisy observations:

$$Y_t = X + \text{noise}$$

- As $t \to \infty$: Y_t becomes pure noise
- As $t \to 0$: recover original data

4.17.2 Stochastic Localization

$$p(X \mid Y_t)$$

defines evolving distributions that interpolate between prior and data.

4.18 Diffusion on Discrete Spaces

- Embed discrete data into continuous space
- Masking / token corruption (language models)
- Define discrete Markov processes directly

4.18.1 Continuous-Time Markov Chain

Generator matrix A:

$$A_{xy} \geq 0, \quad A_{xx} = -\sum_{y \neq x} A_{xy}.$$

Generator:

$$\mathcal{L}f(x) = \sum_{y \neq x} A_{xy}(f(y) - f(x)).$$

Remark 4.18.1. Defines jump process with exponential waiting times.

4.19 Reversing Markov Processes

Theorem 4.19.1. *Let (P_t) be a Markov process with generator A and marginals p_t. The reverse process has generator*

$$B_t(x, y) = A_{yx} \frac{p_t(y)}{p_t(x)}, \quad x \neq y.$$

Remark 4.19.1. Discrete analogue of reverse SDE.

4.19.1 Example: Bit-Flip Process

On $\{-1, 1\}^n$:

- Forward: flip each coordinate randomly
- Reverse: depends on ratio $\frac{p_t(x^{(i)})}{p_t(x)}$

4.20 Stochastic Interpolants

Definition 4.20.1. A **stochastic interpolant** between distributions ρ_0, ρ_1 is:

$$x_t = I(t, x_0, x_1) + \gamma(t)z, \quad z \sim \mathcal{N}(0, I).$$

Example:

$$x_t = (1-t)x_0 + tx_1 + \sqrt{2t(1-t)}\, z.$$

Remark 4.20.1. Generalizes diffusion models to arbitrary endpoints.

4.20.1 Dynamics

- ODE: $dx_t = b_t dt$
- SDE: $dx_t = (b_t + s_t)dt + \sqrt{2\epsilon} dW_t$

4.21 Schrödinger Bridge

4.21.1 Problem

Find a stochastic process matching:

$$X_0 \sim \rho_0, \quad X_T \sim \rho_1$$

while minimizing KL divergence to a reference process.

Definition 4.21.1.

$$\pi^* = \arg\min_{\pi} \mathrm{KL}(\pi \| p_{\mathrm{ref}})$$

subject to endpoint constraints.

4.21.2 Connection to Optimal Transport

- Equivalent to entropy-regularized OT
- Balances transport cost and entropy

4.22 Iterative Proportional Fitting (IPF)

- Alternate between matching forward/backward marginals
- Converges to Schrödinger bridge solution

Remark 4.22.1. Key idea: alternating KL projections.

4.23 Diffusion vs Schrödinger Bridge

- Diffusion: fixed prior $\to$ data
- Schrödinger bridge: arbitrary $\rho_0 \to \rho_1$
- More flexible, but computationally heavier

4.24 Big Picture

- Score-based models learn $\nabla \log p_t$
- Reverse-time dynamics generate samples
- Diffusion = special case of stochastic interpolation
- Schrödinger bridge = optimal interpolation

4.25 Modern Deep Generative Modeling

4.25.1 Unifying View

We consider the problem:

$$\text{Learn } p_{\text{data}}(x) \text{ from samples.}$$

- **Likelihood-based:**
  - Energy-based models (RBM)
  - Variational Autoencoders (VAE)
  - Normalizing flows
- **Non-likelihood-based:**
  - GANs
  - Noise Contrastive Estimation (NCE)
  - Diffusion / score-based models

4.26 Generative Adversarial Networks (GANs)

4.26.1 Formulation

$$\min_{g \in \mathcal{G}} \max_{f \in \mathcal{F}} \Big[\mathbb{E}_{x \sim p_{\text{data}}} \log f(x) + \mathbb{E}_{x \sim p_g} \log(1 - f(x)) \Big]$$

- g: generator, pushes $z \sim p_z$ to $x = g(z)$
- f: discriminator, estimates $P(y = 1|x)$

4.26.2 Optimal Discriminator

$$f^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

4.26.3 Divergence Interpretation

$$\min_g \; 2\,\mathrm{JS}(p_g, p_{\text{data}})$$

Remark 4.26.1. GANs minimize Jensen–Shannon divergence.

4.26.4 Wasserstein GAN

$$\min_g \max_{f \in \mathrm{Lip}(1)} \left[\mathbb{E}_{p_{\text{data}}} f(x) - \mathbb{E}_{p_g} f(x)\right]$$

- Approximates $W_1(p_{\text{data}}, p_g)$
- Requires Lipschitz constraint on f

Remark 4.26.2. More stable than standard GANs when supports are disjoint.

4.26.5 Practical Issues

- Mode collapse
- Training instability (min-max optimization)
- Sensitive to discriminator capacity

4.27 Noise Contrastive Estimation (NCE)

4.27.1 Idea

Reduce density estimation to classification:

$$\ell(f) = \sum_{i=1}^{N} \log f(x_i^{\text{data}}) + \sum_{j=1}^{M} \log(1 - f(x_j^{\text{noise}})).$$

- Data: $x \sim p_{\text{data}}$
- Noise: $x \sim q$

4.27.2 Optimal Classifier

$$f^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + kq(x)}.$$

Remark 4.27.1. Recovers density ratio $\frac{p_{\text{data}}}{q}$.

4.27.3 Key Insight

- Avoids computing partition function
- Choice of q critical (curse of dimensionality)

4.27.4 Telescoping Density Ratio Estimation

Construct intermediate distributions:

$$\frac{p_0}{p_n} = \frac{p_0}{p_1} \cdot \frac{p_1}{p_2} \cdots \frac{p_{n-1}}{p_n}.$$

4.28 Evaluation Metrics

4.28.1 Inception Score (IS)

$$\mathbb{E}_{x \sim p_g} \mathrm{KL}(p(y|x) \| p(y))$$

- High if:
  - confident predictions (low entropy $p(y|x)$)
  - diverse outputs (high entropy $p(y)$)

Remark 4.28.1. Can be gamed (memorization).

4.28.2 Fréchet Inception Distance (FID)

$$W_2^2 = \|\mu - \mu'\|^2 + \mathrm{Tr}(\Sigma + \Sigma' - 2(\Sigma\Sigma')^{1/2})$$

- Compares Gaussian approximations in feature space
- Measures both quality and diversity

4.29 Diffusion Models (Modern View)

4.29.1 Score-Based Perspective

$$s(x) = \nabla \log p(x)$$

- Learn score via score matching
- Sample via Langevin / reverse SDE

4.29.2 Forward / Reverse Processes

- Forward: add noise until $x_T \sim \mathcal{N}(0, I)$
- Reverse: remove noise using learned score

4.29.3 Two-Step View

1. Learn score $s_\theta(x, t)$
2. Simulate reverse dynamics to sample

4.30 Probability Flow and Sampling Refinements

- Reverse SDE (stochastic)
- Probability flow ODE (deterministic)
- Predictor-corrector methods

4.31 Score Matching on Discrete Spaces

Define discrete gradient:

$$\nabla f(x) = (f(x^{(i)}) - f(x))_{i=1}^d.$$

Alternative objective:

$$L_p(q) = \sum_{i=1}^d \mathbb{E}_{x \sim p} \log q(x_i | x_{-i}).$$

Remark 4.31.1. Pseudo-likelihood is discrete analogue of score matching.

4.32 Statistical Efficiency

4.32.1 MLE vs Score Matching

- MLE:
  - statistically optimal
  - computationally hard
- Score matching:

  - computationally efficient
  - statistically suboptimal

4.32.2 Functional Inequalities

- Poincaré inequality $\Rightarrow$ mixing
- Log-Sobolev inequality $\Rightarrow$ fast convergence

4.33 Final Big Picture

- Generative modeling = learning distributions
- Three main paradigms:
  - Optimization (VI, flows, GANs)
  - Sampling (MCMC, diffusion)
  - Hybrid (score-based methods)
- Tradeoffs:
  - Statistical efficiency vs computational tractability
  - Exactness vs scalability

4.34 Calibration and Conformal Prediction

4.34.1 Calibration

Goal: Ensure probabilistic predictions reflect true frequencies.

A predictor $f : \mathcal{X} \to [0, 1]$ is *calibrated* if

$$\mathbb{P}(Y = 1 \mid f(X) = p) = p \quad \forall p \in [0, 1].$$

Remark 4.34.1. Calibration is about *uncertainty correctness*, not accuracy.

- Perfect calibration but poor accuracy: always predict 1/2.
- Good accuracy but poor calibration: overconfident predictions (0 or 1).

4.34.2 Calibration in Deep Learning

- Overparameterized models $\Rightarrow$ overconfidence.
- Minimizing loss $\not\Rightarrow$ calibrated probabilities.

- Post-training (e.g., RLHF) can worsen calibration.

4.35 Proper Scoring Rules

A scoring rule S is *proper* if:

$$\arg\max_{\hat{p}} \mathbb{E}[S(\hat{p}, Y)] = p.$$

Examples:

- Log score: $S(p) = \log p$
- Brier score: $S(p) = -(1-p)^2$

Remark 4.35.1. Proper scoring rules incentivize truthful probabilities.

4.36 Measuring Calibration

4.36.1 Expected Calibration Error (ECE)

$$\mathrm{ECE}(f) = \mathbb{E}\Big[\, |\mathbb{E}[Y \mid f(X)] - f(X)| \,\Big].$$

$$\mathrm{ECE}_2(f) = \mathbb{E}\Big[\, (\mathbb{E}[Y \mid f(X)] - f(X))^2 \,\Big].$$

Remark 4.36.1. In practice, estimated via binning (reliability diagrams).

4.36.2 Decomposition of Error

$$\mathbb{E}[(f(X)-Y)^2] = \underbrace{\mathbb{E}[(f(X)-\mathbb{E}[Y|f(X)])^2]}_{\text{calibration error}} + \underbrace{\mathbb{E}[(\mathbb{E}[Y|f(X)]-Y)^2]}_{\text{intrinsic noise}}.$$

4.37 Calibration Algorithms

Define:

$$\mathrm{calib}(p) = \mathbb{P}(Y = 1 \mid f(X) = p).$$

Construct calibrated predictor:

$$\hat{f}(x) = \mathrm{calib}(f(x)).$$

Theorem 4.37.1. *$\hat{f}$ is calibrated and improves squared loss:*

$$L(\hat{f}) = L(f) - \mathrm{ECE}_2(f).$$

4.37.1 Practical Methods

- Binning / histogram calibration
- Isotonic regression
- Temperature scaling:

$$\hat{f}(x) = \frac{e^{\beta f(x)}}{1 + e^{\beta f(x)}}$$

4.38 Issues in Continuous Case

- Cannot directly estimate $\mathbb{P}(Y = 1|f(X) = p)$
- ECE is discontinuous

Fixes:

- Binning
- Smoothing
- Distance to calibration:

$$d_{\mathrm{CE}}(f) = \inf_{g \in \mathrm{Cal}} \mathbb{E}|f(X) - g(X)|$$

4.39 Extensions

- Multicalibration (fairness across groups)
- Multiclass calibration
- Decision calibration (task-aware)

4.40 Conformal Prediction

4.40.1 Goal

Construct set-valued predictor $T(x)$ such that:

$$\mathbb{P}(Y \in T(X)) \geq 1 - \delta.$$

4.40.2 Nonconformity Score

Define $s(x, y)$ measuring prediction error.

Examples:

- Regression: $s(x, y) = |f(x) - y|$
- Classification: $s(x, y) = 1 - f(x, y)$

4.40.3 Algorithm

Given data $(x_i, y_i)_{i=1}^n$:

$$\tau = \min\left\{t : \sum_{i=1}^{n} \mathbf{1}_{s(x_i,y_i)\leq t} \geq (1-\delta)(n+1)\right\}.$$

Output:

$$T(x) = \{y : s(x, y) \leq \tau\}.$$

4.40.4 Guarantee

Theorem 4.40.1. *For new (x, y):*

$$1 - \delta \leq \mathbb{P}(y \in T(x)) \leq 1 - \delta + \frac{1}{n+1}.$$

4.40.5 High-Probability Version

$$\tau = \text{quantile}_{1-\delta} + \sqrt{\frac{\ln(2/\gamma)}{2n}}$$

4.41 Key Insight

- Calibration $\Rightarrow$ correctness of probabilities
- Conformal prediction $\Rightarrow$ correctness of *sets*
- No distributional assumptions needed (only exchangeability)

4.42 Big Picture

- MCMC / VI: approximate distributions
- Calibration: fix uncertainty of predictions
- Conformal: wrap any model with guarantees